# 实验二　软件设计与建模

## 二手交易系统　设计报告

表 0-1　报告基本信息

| 项目 | 内容 |
| --- | --- |
| 项目名称 | 二手交易系统（参考闲鱼、转转；仅线下交易） |
| 设计者 | 乔轶（学号 241220173） |
| 需求基线 | 实验一 requirements.md（29 条需求，已确认 26 条） |
| 设计产出 | 7 个业务模块 + 16 张 UML 图 + 13 屏 UI 原型 |
| 过程记录 | 架构设计.md、人工修改记录.md、Git 提交记录 |
| 设计基线 | 标签 lab2-design　提交 944f74bb（完整哈希见第七章） |
| 完成日期 | 2026-11-02 |

说明：本报告第四章的图对应交付目录 UML/ 下的同名 .puml 与 .png 文件，第五章的界面图对应 UI 界面/ 目录。所有 UML 图均由 PlantUML 源码渲染，源码随 PNG 一并提交，可按需复现。

# 目录

一、项目概述与设计目标

二、软件的主要功能

三、软件架构与模块划分

四、UML 建模：四类图与模块交互

五、UI 界面设计

六、AI 辅助设计的过程与反思

七、Git 版本记录

八、提交清单与要求对照

# 一、项目概述与设计目标

## 1.1 项目与需求基线

本项目是实验一确定的“二手交易系统”。实验一共形成 29 条需求，其中已确认 26 条（功能需求 21 条、非功能需求 4 条、约束 1 条），已拒绝 3 条（REQ-007 管理员调整搜索排序、REQ-014 审核买卖双方身份、REQ-015 审核取消下单条件）。实验二不再重新获取需求，只把这份需求基线转成可实现的软件设计。

需求基线里对设计影响最大的，是三个核心对象的状态数量与唯一性约束：商品 8 个状态、15 条允许的状态转换；预约 5 个状态，且“同一商品任一时点只允许一条有效预约”；订单 5 个状态，终态不可逆。这些约束不能只写在文档里，必须在架构层面找到“只在一个地方实现、别人绕不过去”的落地方式。

## 1.2 设计目标

本次设计要达到四个目标，后面的模块划分、UML 图和 UI 都是围绕它们展开的：

1. 把 26 条需求分配到互不重叠的模块上，任一状态、任一数据表只有一个写入方。
2. 让 REQ-029 的 15 条状态转换可以被逐条核对、逐条写测试，而不是散落在各个接口里。
3. 在实验一约定的单机单实例环境下（REQ-020），保证并发的“唯一有效预约”和 1 万商品下的查询性能（REQ-022）。
4. 把“预约—确认—下单—交付”这条主线的每个异常分支（超时、取消、下架、驳回）都在界面上说清楚。

## 1.3 设计约束及其影响

表 1-1　非功能约束对设计的影响

| 需求 | 约束 | 对设计的影响 |
| --- | --- | --- |
| REQ-020 | 单机 4C/8G、MySQL 8.0 单实例、应用单实例、不用集群与缓存中间件 | 不做微服务；超时用进程内定时任务；通知用站内消息表，不引消息队列 |
| REQ-022 | 1 万条已发布商品、20 并发、95% 查询 1 秒内返回 | 检索独立成模块，只读、走组合索引；写路径不持有读路径需要的锁 |
| REQ-019 | 密码哈希、越权拒绝、连续失败 5 次锁定 30 分钟 | 认证与鉴权集中在接入层；锁定状态独立表 |
| REQ-027 | 关键操作留痕、可查、不可改、失败不留成功记录 | 审计与业务同事务写入，不用 AOP 事后补记 |
| REQ-028 | 一律逻辑删除；删商品要级联作废预约与订单 | 软删除字段 + 同事务同步事件级联，不用数据库级联 |

## 1.4 本次产出

表 1-2　本次设计的产出

| 产出 | 数量 | 说明 |
| --- | --- | --- |
| 业务模块 | 7 个 + 1 个公共层 | M1 账号与权限、M2 商品管理、M3 商品检索、M4 预约与交易、M5 信用与风控、M6 通知、M7 审计与统计 + M0 公共层 |
| UML 图 | 16 张 | 类图 2、对象图 1、包图 1、组件图 1、部署图 1、用例图 2、活动图 2、状态图 3、时序图 3 |
| UI 界面 | 13 屏 | 买家 6 屏、卖家 3 屏、管理员 4 屏 |
| 设计文档 | 2 份 | 《架构设计.md》《人工修改记录.md》，随仓库提交 |

# 二、软件的主要功能

## 2.1 角色与权限

系统有三类角色，权限边界来自 REQ-002。设计上把“能做什么”固化到模块的接口权限注解上，把“不能做什么”交给状态转换表与守卫来拦，不依赖前端隐藏按钮。

表 2-1　角色权限矩阵

| 能力 | 买家 | 卖家 | 管理员 |
|---|---|---|---|
| 注册 / 登录 | ✓ | ✓ | ✓ |
| 搜索商品、查看详情 | ✓ | ✓ | ✓ |
| 创建商品、填分类与价格 | | ✓（未封禁） | |
| 提交审核 / 修改后重新提交 | | ✓（未封禁） | |
| 审核商品（通过 / 驳回） | | | ✓ |
| 预约商品 | ✓（未封禁，非本人商品） | | |
| 确认预约（24 小时内） | | ✓（本人商品） | |
| 下单 / 取消下单 | ✓（本人订单） | | |
| 下架商品 | | 仅“已发布”“已预约” | 违规时（含“交易中”） |
| 调整价格 | | 仅本人商品 | 不允许 |
| 删除商品 / 预约 | | | 仅确有违规情形 |
| 查看联系方式 | 有效预约或未结束订单期间 | 同左 | 始终可见 |
| 提交举报 / 处理举报 | ✓ | ✓ | ✓（处理） |
| 查看操作记录 | 本人相关 | 本人相关 | 全部 |
| 查看统计报表 | | | ✓ |

## 2.2 功能模块

按业务能力把系统划成 7 个业务模块加 1 个公共层，每个模块对应一个聚合根和一组需求：

表 2-2　功能模块清单

| 模块 | 主要功能 | 对应需求 |
|---|---|---|
| M1 账号与权限 | 注册、登录、BCrypt 哈希、失败计数与锁定、会话、角色鉴权、封禁判定 | REQ-001、002、019、018（拦截） |
| M2 商品管理 | 创建/编辑草稿、分类标签、价格、提交审核、审核通过/驳回、上下架、逻辑删除、商品状态机 | REQ-003、004、005、028、029 |
| M3 商品检索 | 关键词/分类/价格区间/状态组合查询、分页、详情只读视 | REQ-006、022 |

| | | |
|---|---|---|
| | 图 | |
| M4 预约与交易 | 预约、唯一有效预约、确认、下单、取消、完成、超时动作、联系方式可见性判定 | REQ-008～012、024～026、029 |
| M5 信用与风控 | 违约记录（4 类）、6 个月 5 次判定、封禁与解封、举报受理与核实 | REQ-013、018 |
| M6 通知 | 站内消息落库、未读、列表、事务提交后推送 | REQ-009、010、028 |
| M7 审计与统计 | 操作记录写入与查询、三类统计、保留策略 | REQ-016、017、027 |
| M0 公共层 | 统一返回与异常、鉴权拦截、事务模板、定时调度、领域事件、跨模块端口 | 全模块 |

## 2.3 三条主流程

系统的业务可以归结为三条流程，第四章的活动图和状态图分别描述了它们的具体分支：

- 商品上架审核：卖家创建草稿 → 填分类与价格 → 提交审核 → 管理员通过（已发布）或驳回（已驳回，记违约）。驳回后卖家可修改重提；已下架商品也必须重新提交审核。
- 预约交易：买家搜索 → 预约（商品转“已预约”）→ 卖家 24 小时内确认（商品转“交易中”，生成待下单订单）→ 买家 24 小时内下单 → 线下交付 → 订单与商品同时变“已完成”。
- 信用风控：卖家超时未确认、卖家违规下架、买家取消下单或超时未下单、举报核实成立，四类行为各记一次违约；6 个月内满 5 次封禁 6 个月，封禁期间可登录、可搜索，不能上架、不能预约下单。

## 2.4 需求覆盖情况

26 条已确认需求全部有明确的归属模块，没有“无主需求”；反过来，每个模块都能说清自己为哪几条需求负责。下表列出需求与模块的对应关系，第四章再说明这些模块之间如何协作。

表 2-3　需求与模块的对应

| 需求 | 模块 | 需求 | 模块 |
|---|---|---|---|
| REQ-001 注册登录 | M1 | REQ-016 统计 | M7 |
| REQ-002 角色权限 | M1 | REQ-017 长期保存 | M7 |
| REQ-003 商品创建 | M2 | REQ-018 违约封禁 | M5 / M1 |
| REQ-004 提交审核 | M2 | REQ-019 安全性 | M1 |
| REQ-005 下架与调价 | M2 | REQ-020 可靠性 | M0 |
| REQ-006 多条件查询 | M3 | REQ-021 易用性 | 前端 |
| REQ-008 预约 | M4 | REQ-022 性能 | M3 |
| REQ-009 卖家确认 | M4 | REQ-023 范围约束 | 全局 |
| REQ-010 取消与超时 | M4 / M5 | REQ-024 订单状态 | M4 |
| REQ-011 交易完成 | M4 | REQ-025 下单 | M4 |
| REQ-012 联系方式 | M4 | REQ-026 取消下单 | M4 |

| | | | |
|---|---|---|---|
| REQ-013 举报 | M5 | REQ-027 操作记录 | M7 |
| REQ-028 删除级联 | M2 / M4 | REQ-029 状态约束 | M2 / M4 |

## 2.5 明确不做的功能

REQ-023 约定只记录线下交易，因此不接支付、物流与第三方登录；REQ-007、REQ-014、REQ-015 在实验一已标为“已拒绝”，本次设计不给它们留接口、不留入口，避免实现阶段被误当成待办。

# 三、软件架构与模块划分

## 3.1 架构风格

采用模块化单体（modular monolith）加前后端分离：后端是一个 Spring Boot 进程，内部按业务域分包，模块之间只通过接口、领域事件和公共层端口交互，不允许跨模块访问数据表；前端拆成买家 H5、卖家 H5 和管理员后台三个入口，只通过 REST 接口与后端交互。

不选微服务的原因很具体：REQ-020 把部署环境限定在单机单实例、不允许缓存与消息中间件，注册中心、网关、分布式事务、跨服务一致性全部超出这个环境，课程周期内也没有运维投入。模块化单体保留了按域拆分的边界，将来要拆服务时按包平移即可。

## 3.2 模块划分与理由

划分依据有四条：一是按聚合根划分，一个数据表只有一个写入方；二是按读写特征划分，检索是纯读路径、商品是写路径，分开才能各自优化；三是按触发来源划分，违约记录会被四个地方触发，集中在一个模块才能保证口径一致；四是按事务边界划分，必须一起成功或失败的动作放在同一个应用服务里。

初稿（AI 生成）给的是 5 个模块：用户与权限、商品（含检索与审核）、交易（含联系方式）、管理（举报/违约/封禁）、审计与统计。人工审查后改成 7 个，理由如下：

表 3-1　模块划分的人工调整

| 问题 | 初稿的做法 | 改后的做法 | 理由 |
|---|---|---|---|
| 检索与商品写路径混在一起 | M2 商品模块同时管写和查 | 拆出 M3 商品检索，只读、走组合索引 | REQ-022 要求 1 万商品、20 并发下 95% 查询 1 秒返回，读路径不能被写事务的锁拖住 |
| 违约记录有三套来源 | 审核驳回记在商品模块，超时与取消记在交易模块，举报记在管理模块 | 统一到 M5 信用与风控，只暴露 CreditPort.record() | REQ-018 的“6 个月 5 次”必须用一个计数器，三处各写一遍迟早不一致 |
| 通知无归属 | 通知语句直接写在交易模块的业务方法里 | 新增 M6 通知，统一走 NotifyPort | REQ-009、010、028 都要通知，散着写会重复三份代码，改模板要改三处 |
| 审计反向依赖 | 审计模块被业务模块直接调用，统计还要读业务表 | AuditPort 定义在 M0，M7 实现；统计走只读视图 | 避免业务模块依赖审计模块造成循环，同时保证审计与业务同事务 |

有一点容易被忽略：联系方式可见性没有跟着“通知”一起放进 M6，而是留在 M4 交易模块。因为它不是一条消息，而是由“是否存在有效预约或未结束订单”推导出来的交易规则，数据归属在交易模块；如果放进通知模块，通知模块就要反过来读交易数据，形成循环依赖。

## 3.3 模块之间的逻辑关系

模块依赖是一条有向无环图，编译期只允许下表方向：

表 3-2　模块依赖

| 模块 | 依赖 | 依赖方式 |
| --- | --- | --- |
| M1 账号与权限 | M0 | — |
| M2 商品管理 | M0、M1、M5、M6 | 调 CreditPort.record()、NotifyPort.send() |
| M3 商品检索 | M0、M2 | 只读调用 ProductQueryApi |
| M4 预约与交易 | M0、M1、M2、M5、M6 | 查商品状态、记违约、发通知 |
| M5 信用与风控 | M0、M1、M6 | 封禁与举报结果通知 |
| M6 通知 | M0、M1 | 收件人校验 |
| M7 审计与统计 | M0 | 实现 AuditPort；统计读只读视图 |

为了让这张表成立，设计上做了两处反向解耦，这也是模块划分里最关键的两个决定：

- 商品下架要作废该商品下的预约与订单，数据在 M4。如果 M2 直接调用 M4，M4 又要查商品状态，就成了循环依赖。做法是 M2 在事务内发布 ProductOfflineEvent，M4 用同步监听器消费，在同一个事务里作废预约与订单——依赖方向只剩 M2 到事件类型（定义在 M0），事务原子性不受影响。
- 审计记录必须与业务同事务写入，否则会出现“业务回滚了但审计留下了成功记录”，违反 REQ-027。做法是 M0 定义 AuditPort 接口，M7 实现；业务模块只依赖 M0，把操作人、操作类型、变更前后状态作为参数传入，同一事务内落库。切面（AOP）方案拿不到业务的前后状态，也无法感知事务是否回滚，所以不用。

## 3.4 模块之间的交互方式

表 3-3　模块交互方式

| 方式 | 使用场景 | 一致性保证 |
| --- | --- | --- |
| 应用服务同步调用 | 校验、状态转换、记违约 | 同一事务，失败一起回滚 |
| 同事务同步事件 | 商品下架 → 作废预约与订单；商品删除 → 级联 | 同一事务，原子 |
| 事务提交后事件 | 通知推送（未读数刷新、SSE） | 提交后才推，回滚不推 |
| 定时任务 | 预约 24h 未确认、订单 24h 未下单 | 分批取 + 幂等更新 |
| 只读查询 | 商品列表、统计、操作记录 | 不加锁，走索引 |

需要强调通知的两段式设计：通知内容与业务状态在同一事务内落库，保证“预约成功”与“通知买家”同生共死；推送（未读红点、SSE）放在事务提交之后。这样即使进程在提交后崩溃，消息仍在库里，用户下次打开消息中心能看到；反过来，如果只在提交后落库，崩溃就会丢消息。

## 3.5 关键设计决策

表 3-4　关键设计决策（ADR）

| 编号 | 决策点 | 方案 | 否掉的方案与原因 |
| --- | --- | --- | --- |

| | | | |
|---|---|---|---|
| ADR-01 | 模块划分 | 7 个业务模块 + 公共层 | 5 个模块：检索无法独立优化，违约口径会漂 |
| ADR-02 | 商品状态机 | 15 条状态转换表 + 守卫链 | 状态模式：8 个类各写一套方法，REQ-029 逐条验收时对不上表 |
| ADR-03 | 唯一有效预约 | active_key 生成列 + 唯一索引 | 行锁：会挡住同商品详情查询，与 REQ-022 冲突；乐观锁：插入场景重试易双插 |
| ADR-04 | 超时处理 | (state, deadline) 索引 + 每批 200 条 + 幂等 UPDATE | 全表扫描无索引；MySQL EVENT 把业务写进数据库难调试 |
| ADR-05 | 通知 | 同事务落库 + 提交后推送 | 业务方法内同步写：回滚后仍有消息；只提交后落库：崩溃丢消息 |
| ADR-06 | 跨模块级联 | 同事务同步事件 | 直接调用 M4：形成循环依赖 |
| ADR-07 | 联系方式可见性 | 实时推导，不建表 | 可见性表：七条状态变化路径都要同步，漏一条就泄露或误藏 |
| ADR-08 | 操作记录 | 显式调用 AuditPort，同事务写入 | AOP 切面：拿不到前后状态，失败也可能留记录 |
| ADR-09 | 逻辑删除 | deleted 字段 + 仓储统一条件；完成订单不可删 | 物理删除违反 REQ-028 |
| ADR-10 | 鉴权 | Session + 拦截器 + @RequireRole；封禁在写操作前拦 | JWT 需要额外密钥管理，单实例无必要 |

## 3.6 三个关键机制的落地方式

**下面三个机制是这份设计的核心，报告后面第四章的图也是围绕它们画的。**

### 3.6.1 状态机：一张表 + 一串守卫

REQ-029 的验收方法是“对允许的转换逐条构造操作，对禁止的转换逐条构造操作”。因此状态转换必须是可枚举、可对照的数据，而不是分散在各接口的 if。设计把 8 个状态、15 条转换写成静态表，查表命中才执行；不在表里的组合一律抛 IllegalStateTransition，状态不变。守卫按固定顺序执行：

表 3-5　守卫链

| 守卫 | 检查内容 | 失败结果 |
|---|---|---|
| StateGuard | 当前状态是否为规则里的 from | 409 状态已变更 |
| RoleGuard | 操作人角色是否具备该动作权限 | 403 越权 |
| OwnerGuard | 卖家只能操作自己的商品；买家不能预约自己的商品 | 403 非本人商品 |
| BanGuard | 被封禁用户不能上架、改价、预约、下单 | 403 封禁中 |
| ReasonGuard | 管理员下架/删除必须填写依据 | 400 缺少理由 |

这样做的好处很直接：REQ-029 的每一条“允许”可以对着表写一条单元测试，每一条“禁止”也可以对着表写一条；新增动作时只改表，不改散落的 if。

### 3.6.2 唯一有效预约：把约束交给数据库

REQ-008 要求同一商品任一时点只有一条有效预约，并发预约只有一个成功。初稿用的是行锁（SELECT ... FOR UPDATE），人工审查时否掉了：锁的是商品行，同商品详情查询会和预约事务抢同一行锁，而 REQ-022 恰恰要求查询在并发下仍能 1 秒返回。

改后的方案是在预约表上加一个生成列，有效状态时等于商品 ID，终态为 NULL，再对它建唯一索引：

```
ALTER TABLE t_reservation
  ADD COLUMN active_key BIGINT GENERATED ALWAYS AS (
    CASE WHEN state IN ('WAIT_SELLER','CONFIRMED') THEN product_id END) STORED,
  ADD UNIQUE KEY uk_reservation_active (active_key);
```

并发时第二个请求在 INSERT 上直接撞唯一键，抛 DuplicateKeyException 转成 409；不需要先查后插，也不给商品行加锁。预约进入终态后 active_key 变 NULL，NULL 不参与唯一性，同一商品可以再次被预约。即使两个买家都读到了“商品已发布”的旧状态，数据库仍然是最终裁判，这正好覆盖 REQ-008 里“旧页面继续预约”的异常场景。

### 3.6.3 超时与通知：分批扫描、幂等处理、两段式通知

预约 24 小时未确认、订单 24 小时未下单都要由系统自动处理，并且要通知买家、卖家和管理员。设计给两个截止时间字段建了 (state, deadline) 索引，任务每 30 秒取 200 条到期记录，每条用带状态条件的 UPDATE 更新：影响行数为 1 才算处理成功，为 0 说明已被别的流程处理过，直接跳过。

这样任务重跑、两批数据重叠都不会重复记违约、重复发通知。通知内容在同一事务里落库，提交后再推送，兼顾了“不漏”和“不误发”。

## 3.7 数据存储设计

数据落在 MySQL 8.0 单实例里，共 11 张表。关键约束如下：

表 3-6　数据表设计

| 表 | 用途 | 关键字段 / 约束 |
|---|---|---|
| t_user | 用户（三角色同表） | username 唯一；password_hash（BCrypt）；role；status；banned_until |
| t_login_attempt | 登录失败计数 | user_id；fail_count；locked_until |
| t_product | 商品 | seller_id；price；category_tag；state；version；deleted |
| t_product_image | 商品图片 | product_id；url；sort |
| t_reservation | 预约 | active_key 生成列 + 唯一索引；confirm_deadline；state；version |
| t_order | 订单 | order_no 唯一；reservation_id；place_deadline；state |
| t_violation | 违约记录 | user_id；type；biz_type；biz_id；occurred_at |
| t_ban | 封禁记录 | user_id；start_at；end_at；status |

| | | |
|---|---|---|
| t_report | 举报 | reporter_id；target_type；target_id；status |
| t_notification | 站内通知 | user_id；type；biz_type；biz_id；is_read |
| t_operation_log | 操作记录 | operator_id；action；target_id；before_state；after_state |

除唯一索引外，还建了三类索引支撑验收条件：检索组合索引 (state, category_tag, price) 对应 REQ-022；(state, confirm_deadline) 与 (state, place_deadline) 对应超时扫描；(user_id, occurred_at) 对应“6 个月内违约次数”的滑动窗口统计。商品、预约、订单一律逻辑删除，查询统一带 deleted = 0；已完成订单在服务层硬校验不可删除（REQ-028）。

## 3.8 部署设计

部署与 REQ-020 的约定环境一致：单机、应用单实例、MySQL 单实例，不启用集群、Redis 与消息队列。Nginx 负责静态资源与反向代理，定时任务与应用同进程，商品图片存本地目录。

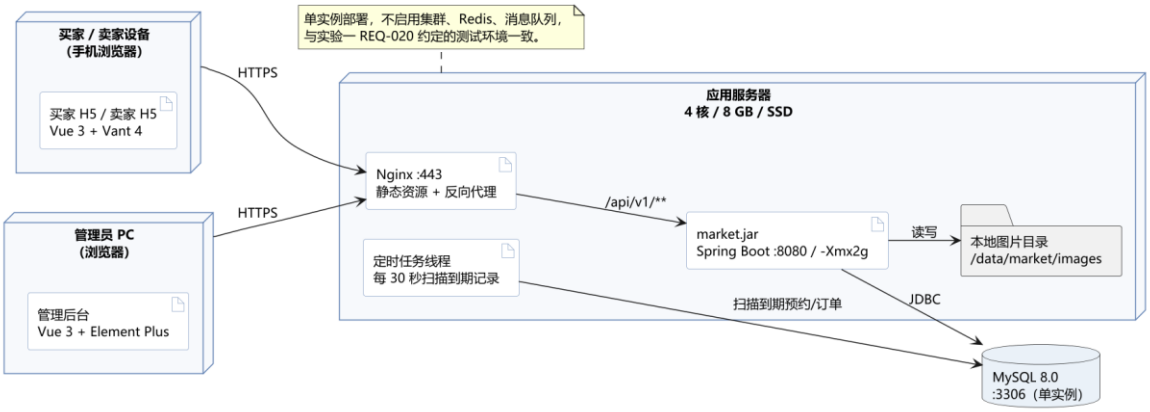



图 3-1　部署图（单机单实例）

# 四、UML 建模：四类图与模块交互

## 4.1 图的选择与分类

实验要求至少四张图：类图必画，结构图、行为图、交互图每类至少一张。本项目实际画了 16 张，选择依据是“这张图能不能回答设计里的一个具体问题”，而不是为了凑数量：

表 4-1　UML 图的分类与用途

| 类别 | 图 | 数量 | 回答的问题 |
|---|---|---|---|
| 结构图 | 类图（领域模型） | 1 | 业务对象有哪些、字段和关系是什么、唯一约束落在哪 |
| 结构图 | 类图（服务与模式） | 1 | 状态机、守卫、端口这些设计元素怎么组织 |
| 结构图 | 对象图 | 1 | 某一时刻一个商品、两条预约、一个订单的具体样子 |
| 结构图 | 包图 | 1 | 后端怎么分包、允许哪些依赖、为什么不会成环 |
| 结构图 | 组件图 | 1 | 7 个模块各自负责什么、彼此怎么调用 |
| 结构图 | 部署图 | 1 | 进程、数据库、文件放在哪台机器上 |
| 行为图 | 用例图 | 2 | 三类角色分别能做什么（按买卖双方 / 管理端拆开） |
| 行为图 | 活动图 | 2 | 上架审核、交易主流程的分支与循环 |
| 行为图 | 状态图 | 3 | 商品 8 态、预约 5 态、订单 5 态的合法迁移 |
| 交互图 | 时序图 | 3 | 并发预约、确认下单交付、超时扫描的调用顺序 |

初稿的类图是一张 28 个类的大图，导出后 5890×2166 像素，A4 上根本看不清；用例图 22 个用例被拉成 843×2658 的细长条。人工修改时把类图拆成“领域模型”和“服务与模式”两张，用例图按角色视角拆成两张——拆的依据是“一张图只回答一个问题”，细节放到表和正文里。

## 4.2 结构图

### 4.2.1 类图（一）：领域模型

这张图只放实体、枚举、关系和领域方法，不画服务。重点是三类对象的归属和多重性：一个卖家多个商品；一个商品多条预约，但只有一条有效；一条被确认的预约生成一个订单；一个用户多条违约记录、封禁记录、通知。预约上的 activeKey 直接标出了唯一约束的落点。

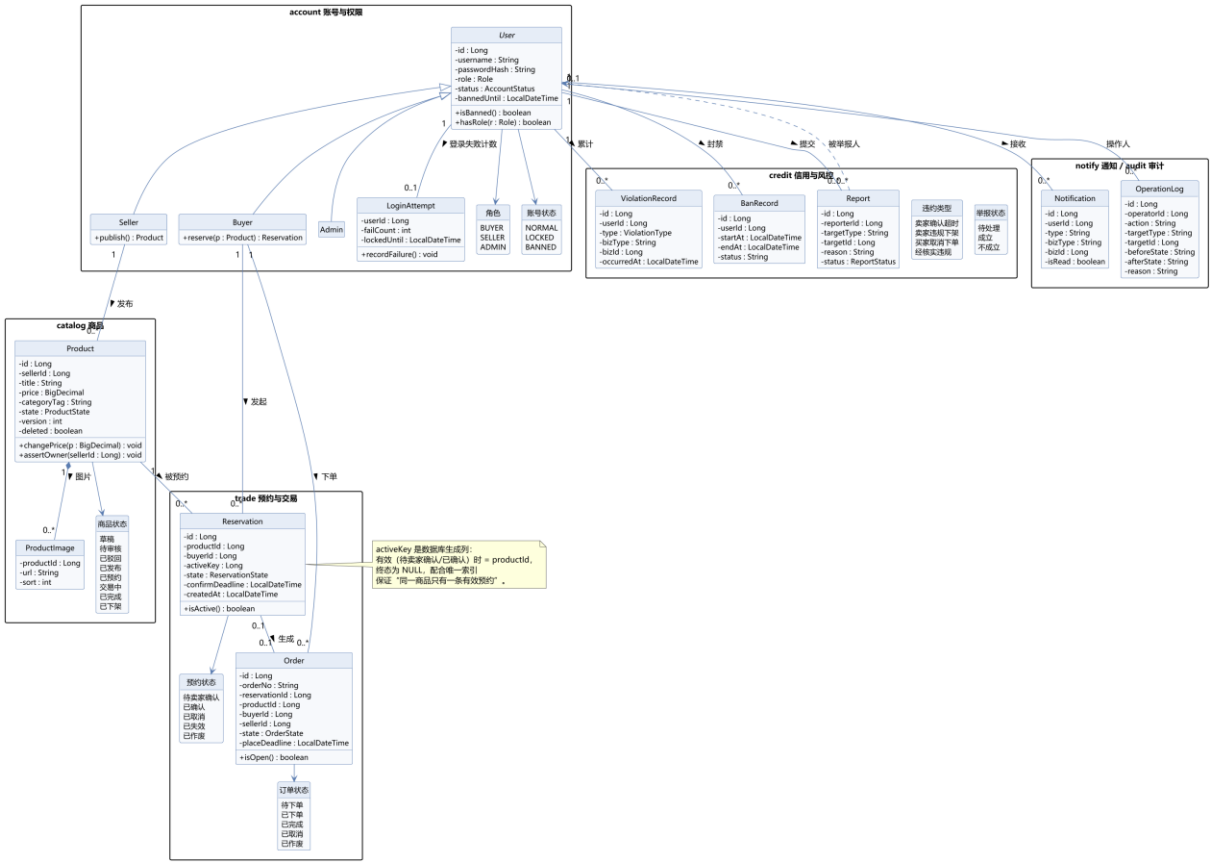



图 4-1　类图（领域模型）

几个设计要点：User 用单表继承（role 区分角色），Buyer/Seller/Admin 只是领域视角，不额外建表；ProductState、ReservationState、OrderState 用枚举而不是字符串，状态取值与需求文档一一对应；activeKey 是数据库生成列，在类图上显式画出来，是为了让“唯一有效预约”这件事在图上有据可查。

### 4.2.2 类图（二）：服务与设计模式

这张图回答“规则写在哪”。应用服务按模块分布，状态机持有转换规则并依次执行守卫，跨模块能力通过四个端口接口暴露，避免服务之间直接依赖实现类。

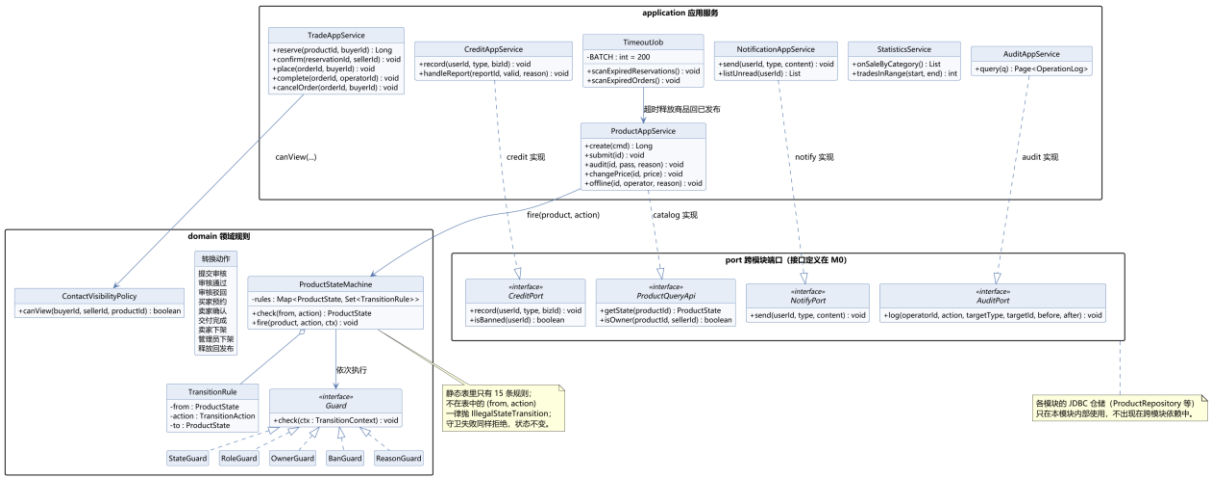



图 4-2　类图（应用服务、状态机与端口）

ProductStateMachine 与 TransitionRule 是组合关系，Guard 是接口、五个实现按顺序执行；ProductQueryApi、CreditPort、NotifyPort、AuditPort 四个端口分别由商品、信用、通知、审计四个模块实现，业务服务只依赖接口。图中不再画仓储接口，因为仓储是模块内部实现，跨模块依赖里不应该出现它。

### 4.2.3 对象图

对象图用来说明唯一索引在真实数据上的效果：同一个商品下有一条“已确认”的预约（activeKey = 1001，占着唯一索引）、一条“已失效”的预约（activeKey 为 NULL，不参与唯一性）和一个待下单订单。这条失效预约的存在恰恰证明了“终态让位、商品可再次被预约”。

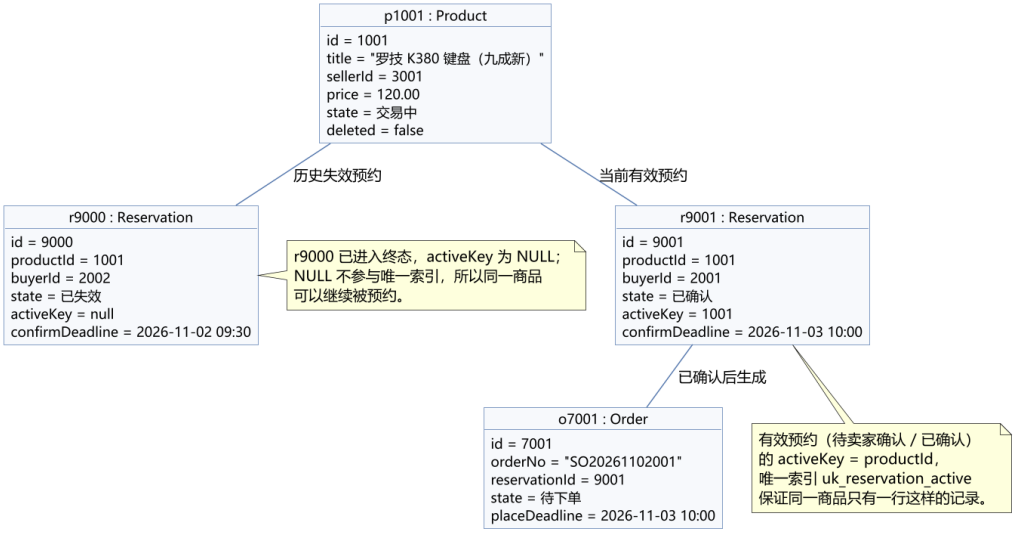



图 4-3　对象图（某商品在某一时刻的快照）

### 4.2.4 包图

包图描述后端的分包结构与依赖规则：每个模块内部固定 api / application / domain / infrastructure 四层，模块之间只允许调用对方的 api，跨模块能力走 M0 的端口与事件；M2 不依赖 M4，M7 不被业务模块反向依赖。

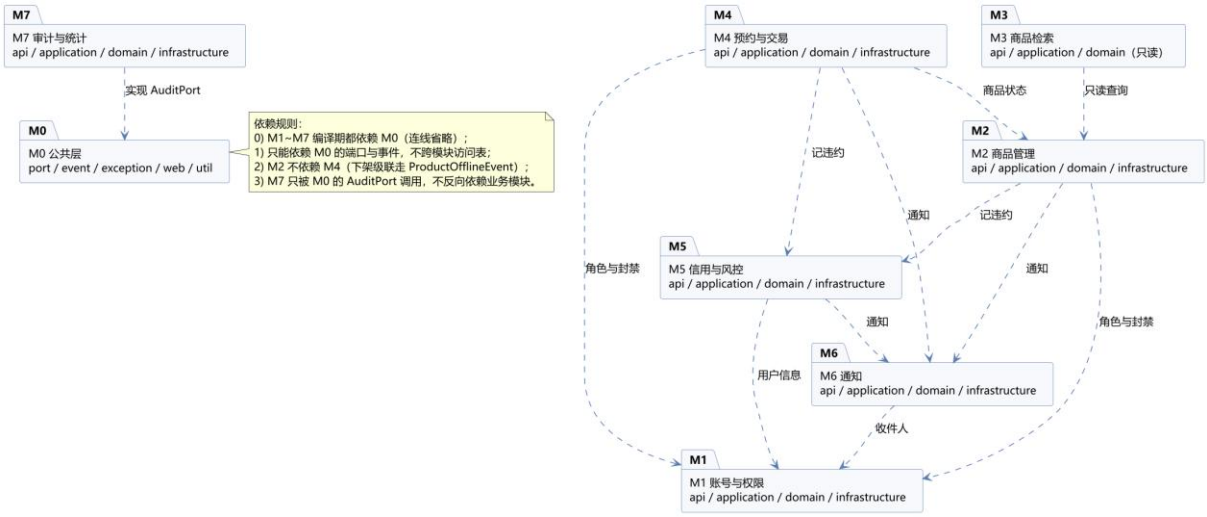



图 4-4　包图（后端模块包与允许的依赖）

### 4.2.5 组件图

组件图站在“部署单元”看系统：三个前端入口、一个接入层、7 个业务组件、一个定时任务、一个 MySQL 和一个图片目录。箭头标出了模块之间的真实调用：交易查商品状态、商品记违约、交易发通知、信用发封禁通知。

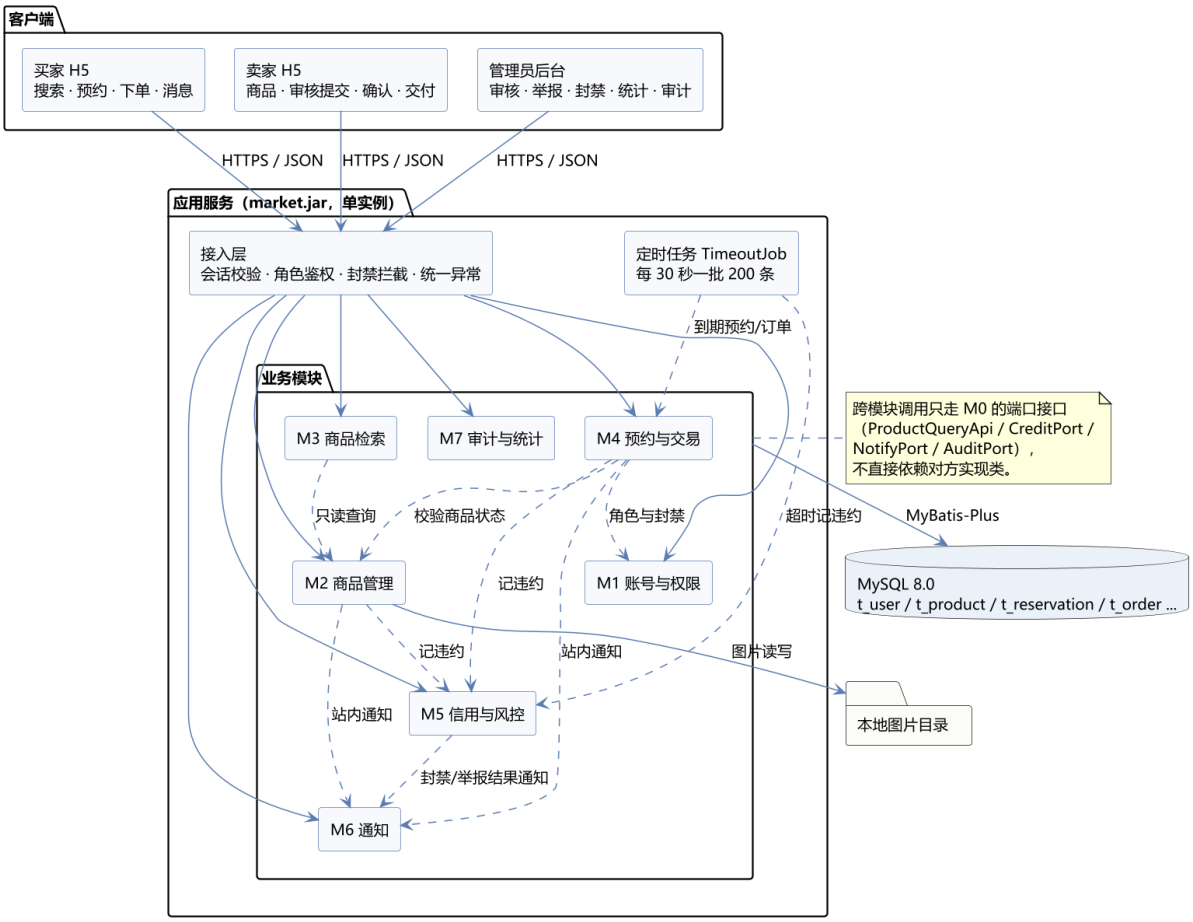



图 4-5　组件图（7 个业务模块及其交互）

### 4.2.6 部署图

部署图见 3.8 节的图 3-1，这里不重复。它要表达的核心是“单实例”：没有集群、没有缓存、没有消息队列，定时任务与应用同进程，因此单实例下不会出现两个任务同时处理同一条记录的问题；将来若扩成多实例，幂等更新仍然能兜住。

## 4.3 行为图

### 4.3.1 用例图（买家与卖家）

按角色视角拆成两张后，买家与卖家这张只保留 11 个用例，主干一眼能看完。所有用例都需要先登录，为避免每条连线都拉一根 include 虚线，图下用注释统一说明。

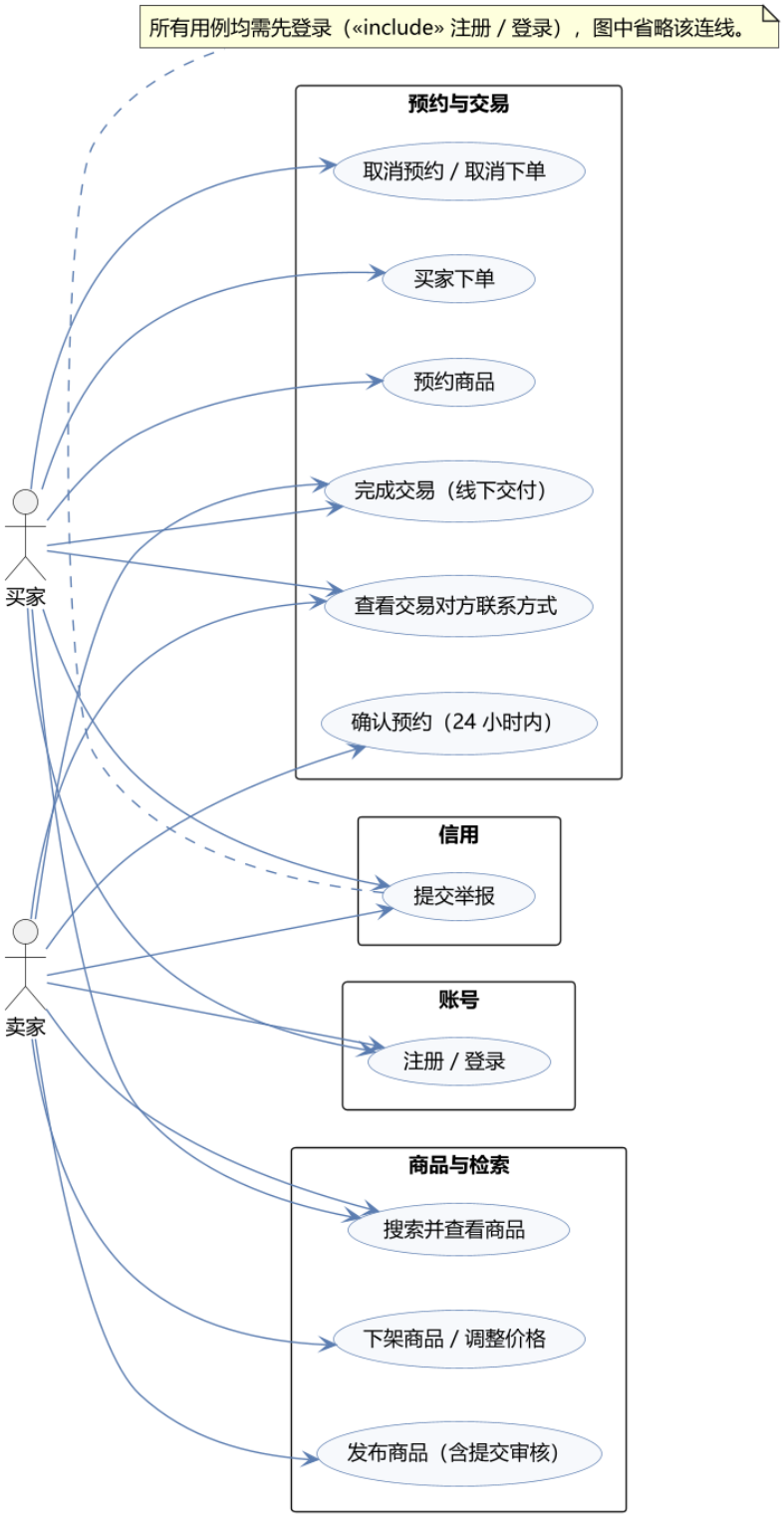



图 4-6　用例图（买家与卖家）

### 4.3.2 用例图（管理员与系统）

管理端 6 个用例加 1 个系统用例。系统计时器作为参与者出现，是因为“超时自动失效”不是任何用户触发的，但又确实是系统对外承诺的行为（REQ-010、REQ-025），画出来才能和需求对上。

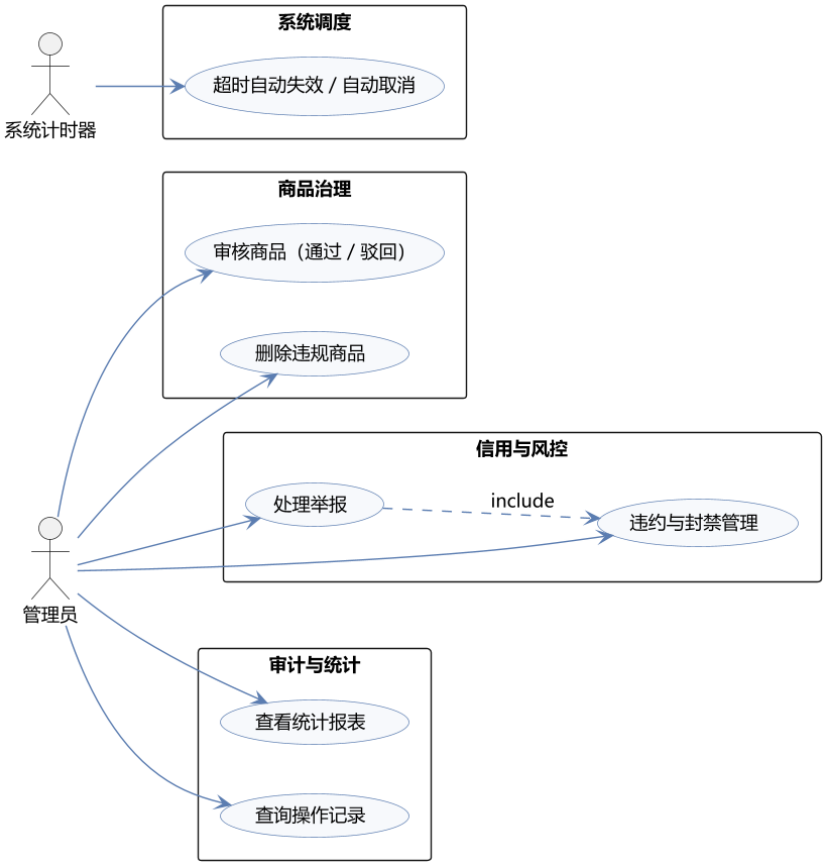



图 4-7　用例图（管理员与系统）

### 4.3.3 活动图：商品上架审核

这张图把 REQ-003、004、018 串起来：提交审核前的守卫判断、管理员通过/驳回的分支、驳回后记违约并判断是否封禁、卖家修改后重新提交的循环。泳道把卖家、系统、管理员的动作分开，避免“谁做了这一步”说不清。

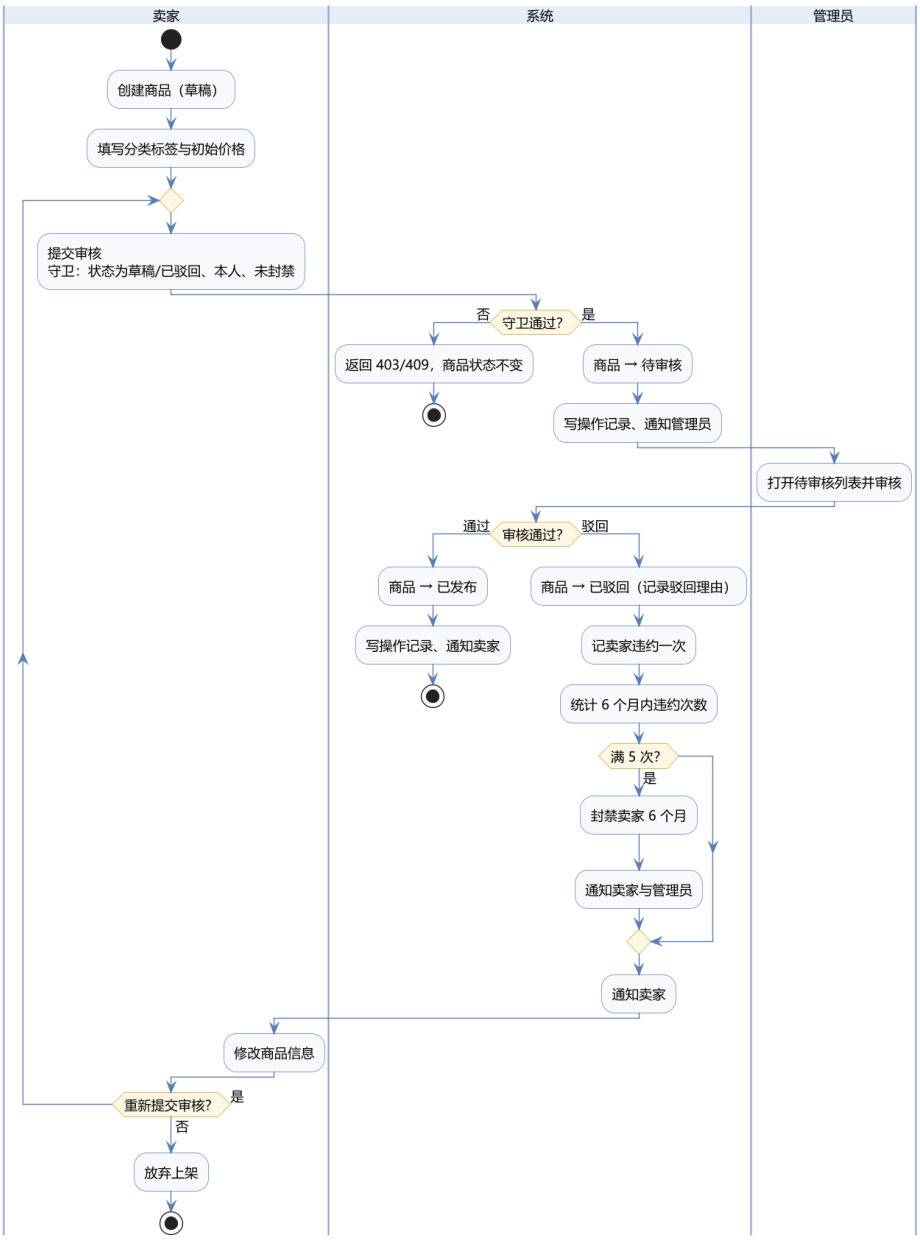



图 4-8　活动图（商品上架审核）

### 4.3.4 活动图：交易主流程

交易主线有四个分支点：预约时的唯一索引冲突、卖家是否在 24 小时内确认、买家是否在 24 小时内下单、线下交付完成。每个分支的失败路径都写了后续动作（改状态、记违约、发通知），与 REQ-008～011、024～026 一一对应。

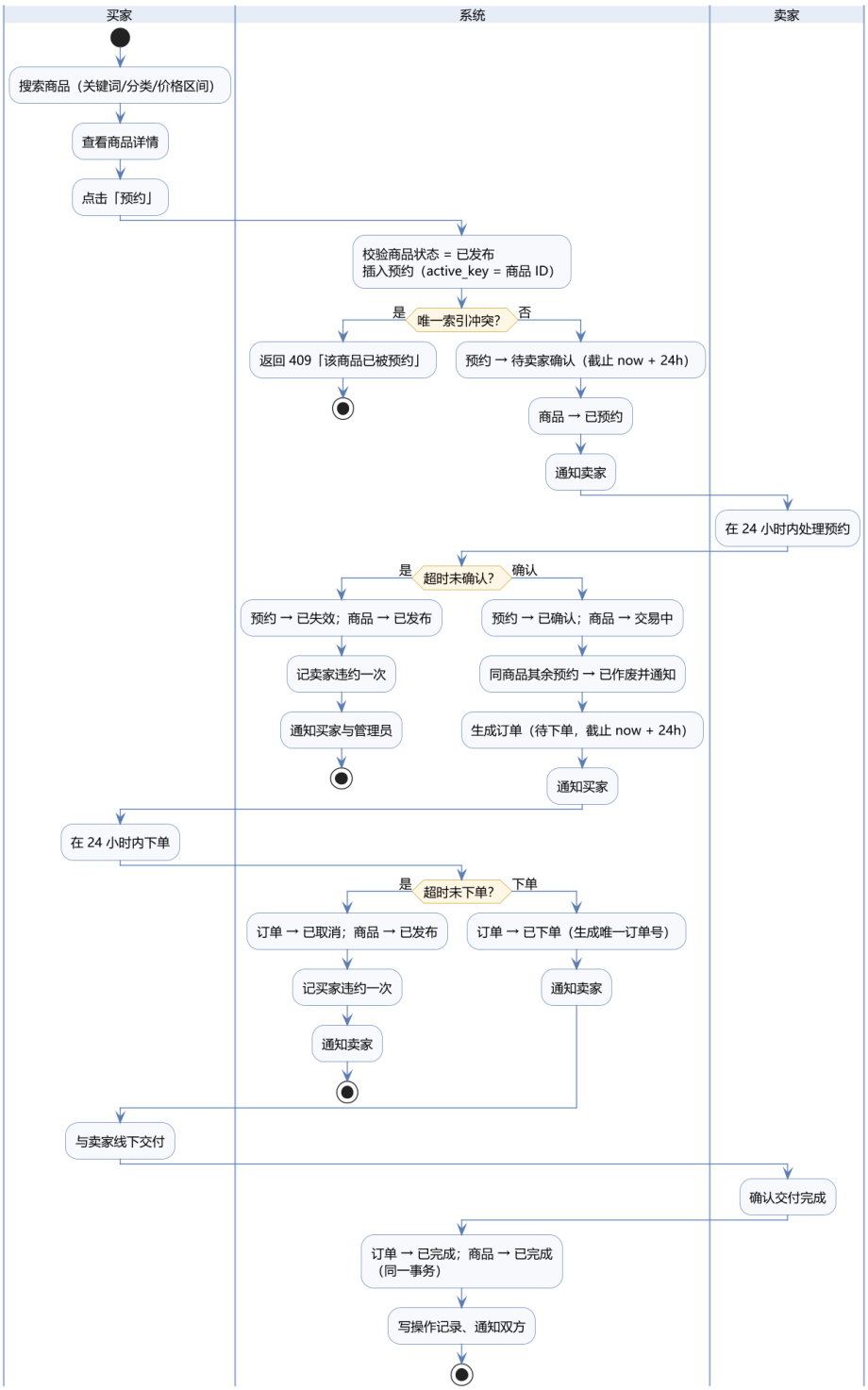



图 4-9　活动图（预约—确认—下单—交付）

### 4.3.5 状态图：商品

商品 8 个状态、15 条允许转换全画出来，边上注明触发者（卖家、买家、管理员、系统）。图注里列了四类禁止转换，它们和图上的箭头是互补的：图上画“能走的”，注释说明“不能走的”由状态表兜底。

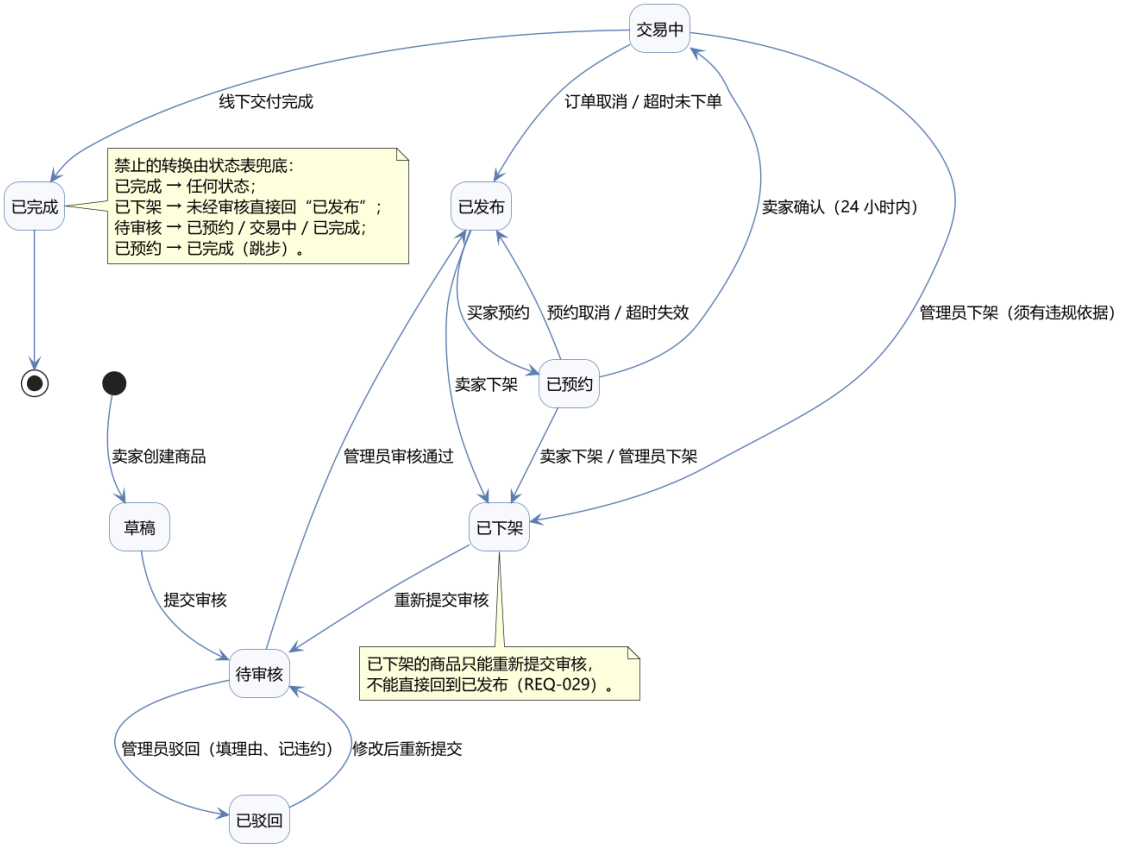



图 4-10　状态图（商品，8 个状态）

### 4.3.6 状态图：预约与订单

预约和订单各自只有 5 个状态，单独成图比挤在商品图里清楚。预约的重点是“终态不能再回到已确认”；订单的重点是“终态不可逆”和“订单与商品的联动”。

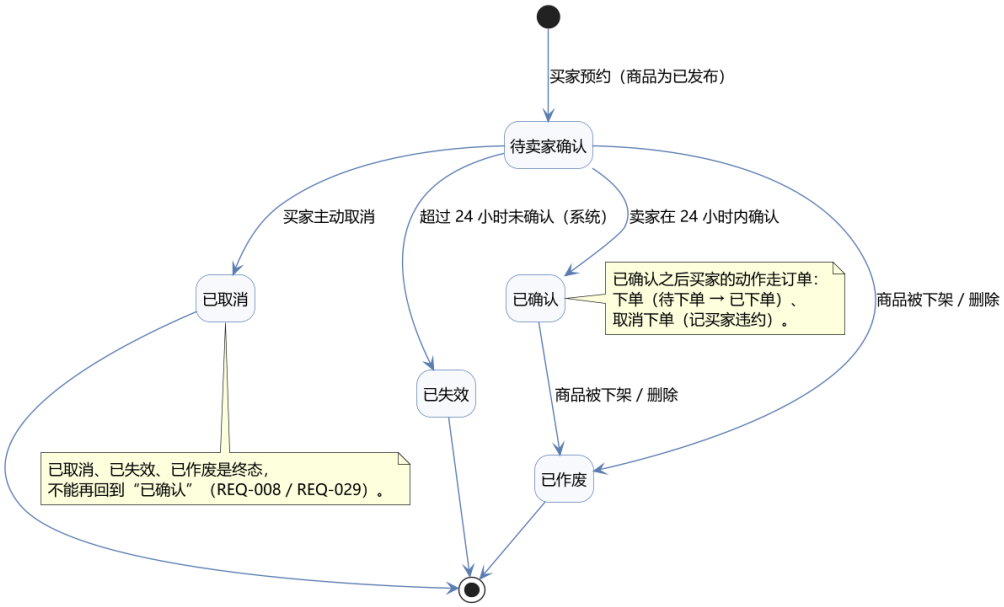



图 4-11　状态图（预约，5 个状态）

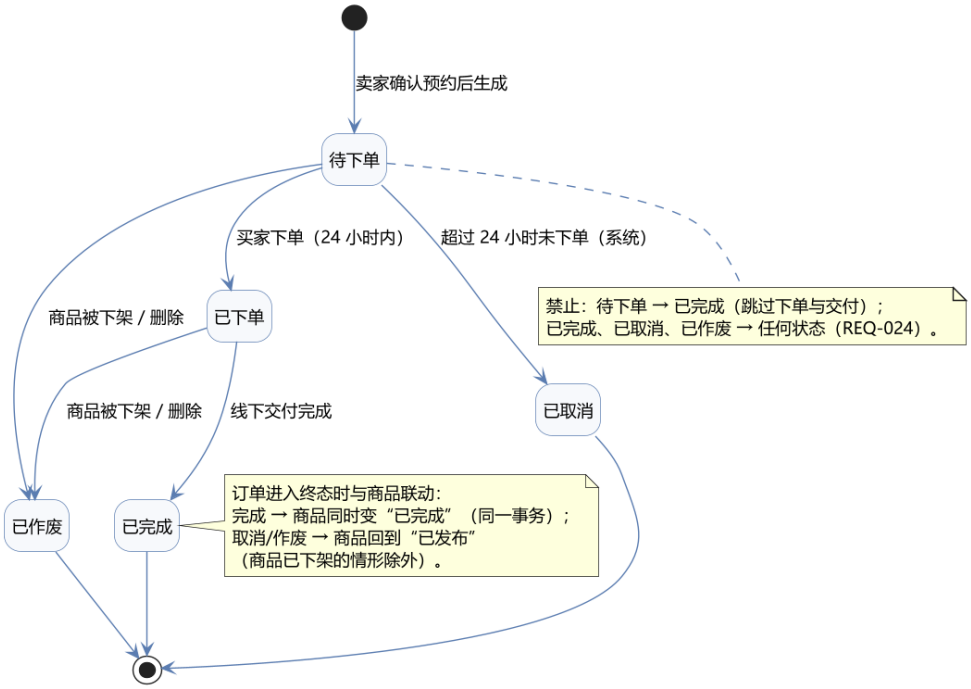



图 4-12　状态图（订单，5 个状态）

## 4.4 交互图

### 4.4.1 时序图：并发预约

这张图是“唯一索引”方案最直接的证据：两个买家几乎同时提交，都读到“商品已发布”的旧状态，第一条 INSERT 占用唯一索引成功，第二条在数据库层被拒绝，事务回滚、不写审计、不发通知。整个过程没有给商品行加锁，因此商品详情查询不受影响。

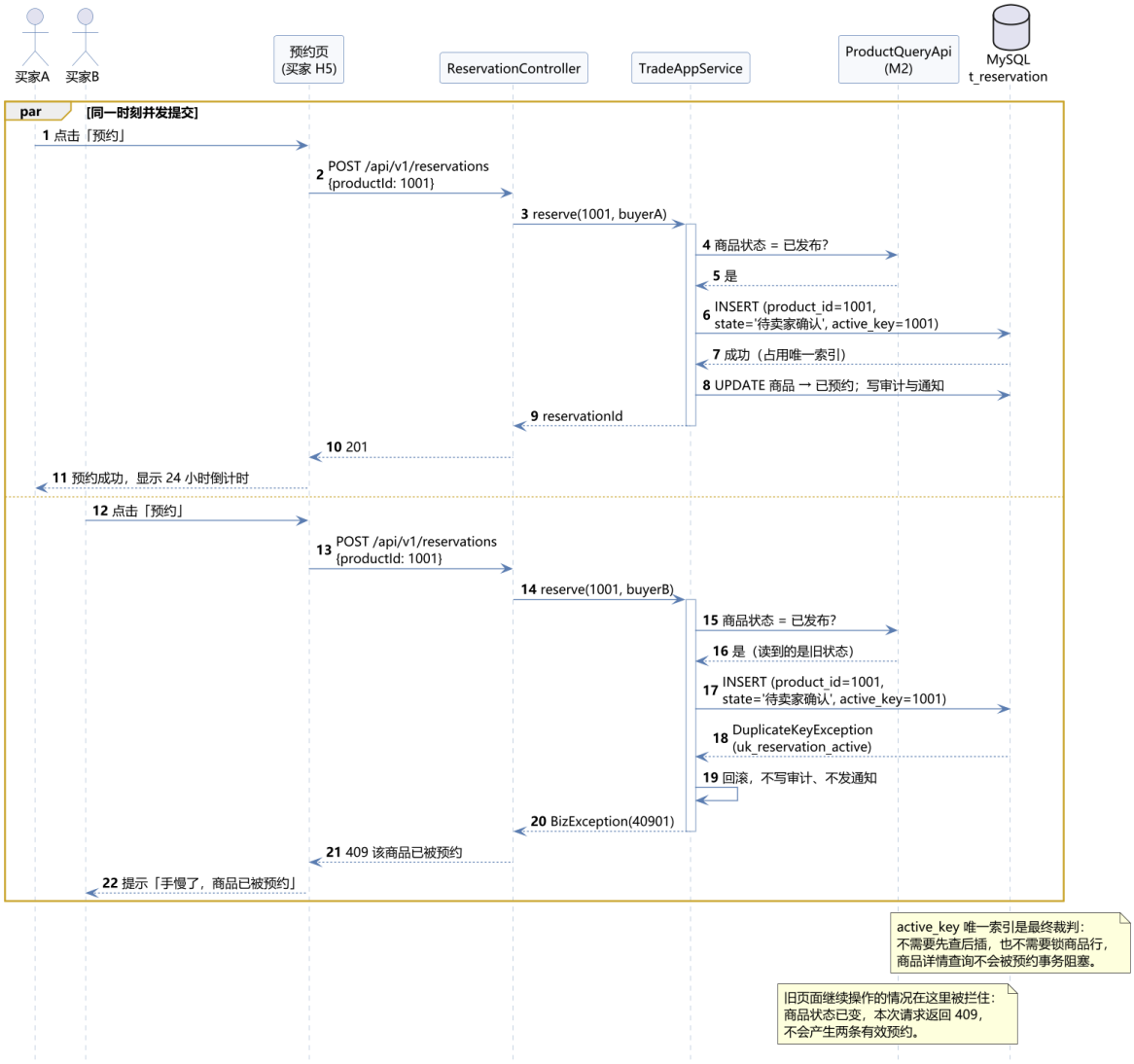



图 4-13　时序图（并发预约与唯一索引）

### 4.4.2 时序图：确认—下单—交付

这张图把三个阶段放在一起：卖家确认（预约转已确认、商品转交易中、其余预约作废、生成待下单订单）、买家下单、卖家确认交付完成（订单与商品在同一事务内变已完成）。每个阶段的审计与通知都在事务内落库，提交后才推送。

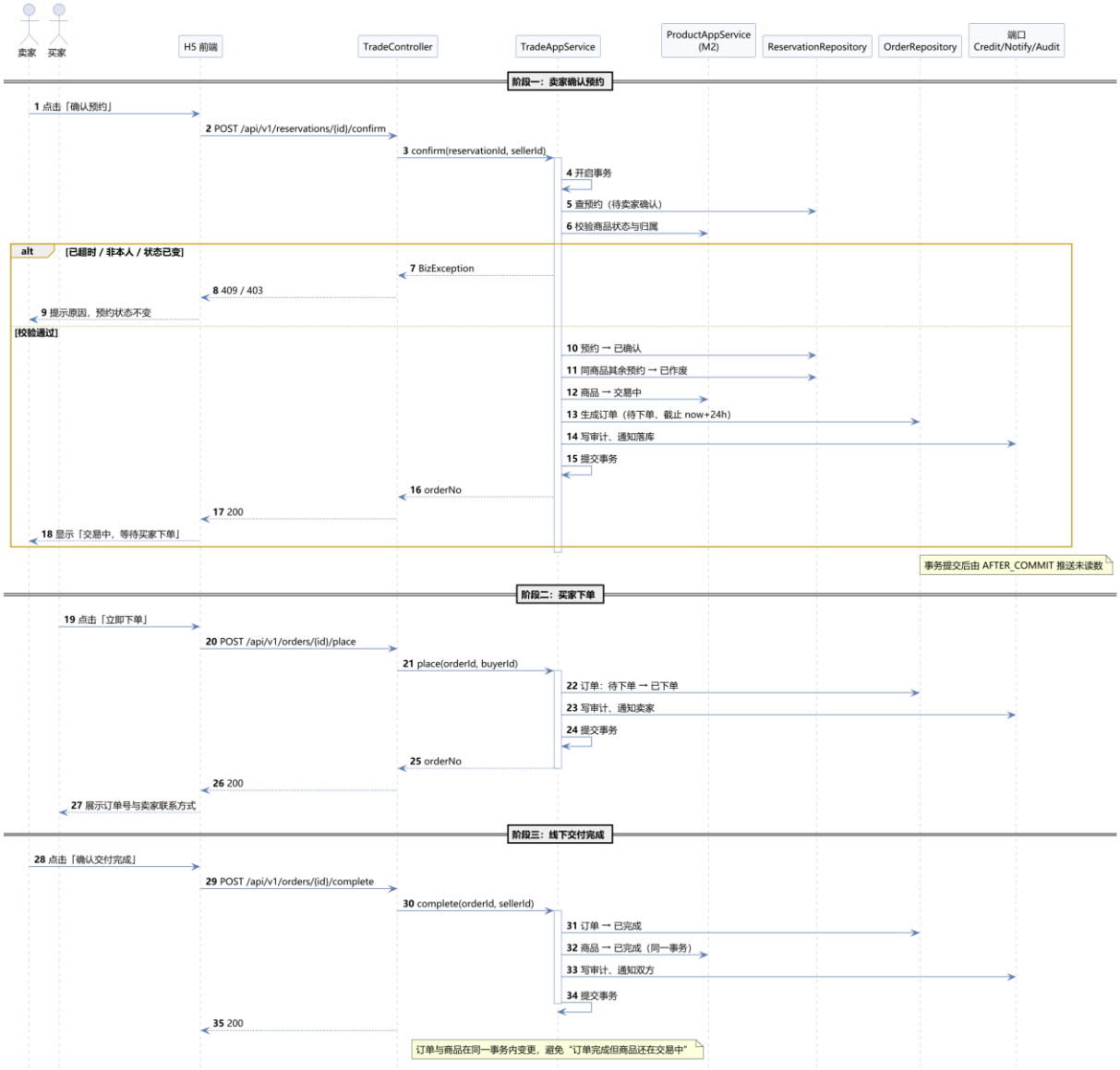



图 4-14　时序图（确认、下单、交付完成）

### 4.4.3 时序图：超时扫描

这张图解释了“分批 + 幂等”的具体做法：按索引取 200 条，逐条用带状态条件的 UPDATE 更新，以影响行数判断是否由本次任务处理；处理成功才记违约、写审计、发通知；取满 200 条继续取下一批。

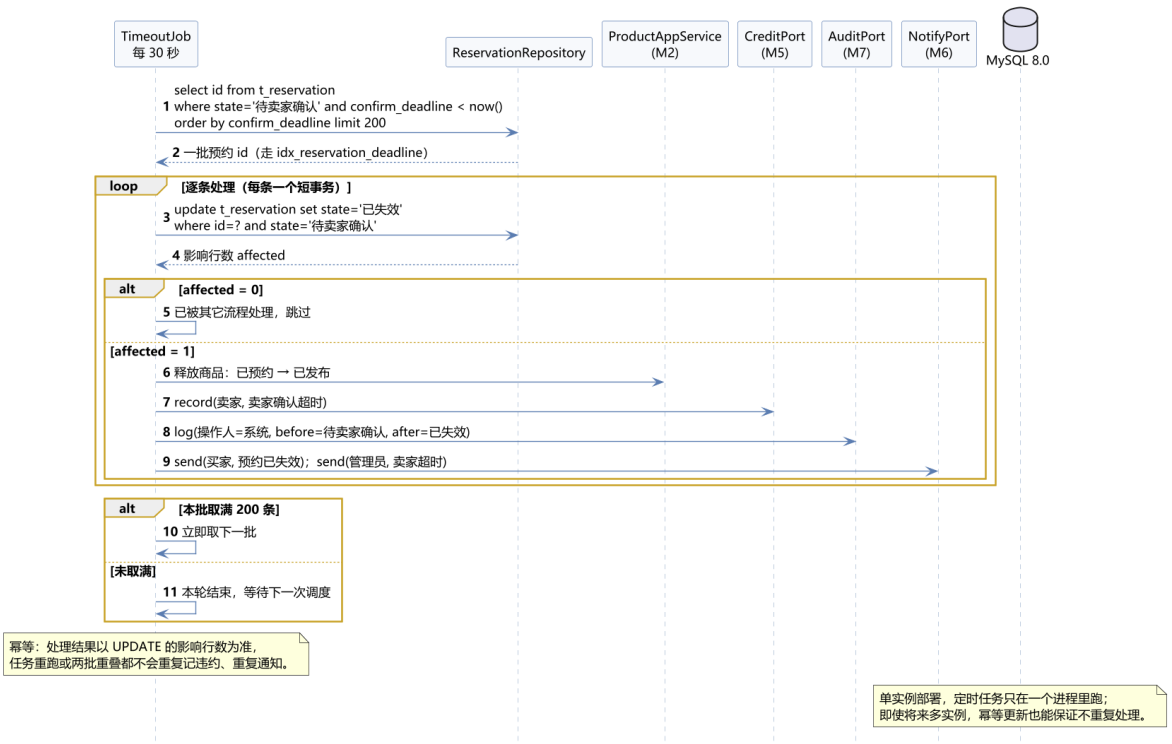



图 4-15　时序图（超时扫描与幂等处理）

## 4.5 模块交互的完整链路

把上面的图串起来，可以看出三条链路如何在模块之间流转：

表 4-2　三条主要链路经过的模块

| 链路 | 经过的模块 | 对应图 |
|---|---|---|
| 预约链 | 买家 → M4 预约 → M2 查商品状态 → 唯一索引兜底 → M7 审计 → M6 通知落库 → 提交后推送 | 图 4-9、图 4-13 |
| 审核链 | 卖家 → M2 状态转待审核 → 管理员审核 → 通过转已发布 / 驳回调 M5 记违约 → M6 通知 | 图 4-8、图 4-10 |
| 超时链 | M0 定时任务 → M4 置失效/取消 → M2 释放商品 → M5 记违约 → M7 审计 → M6 通知 | 图 4-15、图 4-11、图 4-12 |
| 下架链 | 卖家/管理员 → M2 下架 → 发布 ProductOfflineEvent → M4 作废预约与订单 → M5 视情况记违约 → M6 通知 | 图 4-5、图 4-10 |

# 五、UI 界面设计

界面按角色分成两端：买家与卖家在手机上完成交易，用移动端 H5（390×844）；管理员在电脑上批量审核、处理举报、看统计，用 PC 后台（1440×900）。两端共用一套色彩与状态语义，但信息密度不同：手机上每次只做一件事，后台一屏能看到十几条记录。

## 5.1 设计规范

表 5-1　两端的设计规范

| 项目 | 移动端（买家 / 卖家） | 管理后台 |
| --- | --- | --- |
| 画布 | 390 × 844（iPhone 14 逻辑尺寸） | 1440 × 900 |
| 主色 | #FF5A1F（预约、下单等主操作） | 同左（主操作按钮），侧栏 #1B2029 |
| 背景 / 卡片 | #F5F6F8 / #FFFFFF，圆角 14px | #F5F6F8 / #FFFFFF，圆角 12px |
| 字号 | 标题 16、正文 13–14、说明 11–12 | 页面标题 20、表格 12–13、辅助 11 |
| 状态色 | 在售绿、已预约橙、交易中蓝、已驳回红、已失效灰 | 同左，用于表格状态标签 |
| 导航 | 底部 4 个 Tab | 左侧 6 项菜单 + 顶部面包屑 |

状态色是两端统一的：同一个状态在手机上和在后台里颜色一致，评审时不需要重新认一遍。另外所有“不可操作”的按钮都不隐藏，而是置灰并给出原因——隐藏按钮会让用户以为系统坏了，置灰加一句话能直接对应到需求里的拒绝理由。

## 5.2 买家端界面




图 5-1　登录 / 注册




图 5-2　买家首页与多条件搜索

登录页只保留手机号、密码两个输入和一个主按钮，把“连续失败 5 次锁定 30 分钟”写在按钮下方，对应 REQ-019 的锁定规则；注册时选择买家或卖家角色，对应 REQ-001。

首页是检索的入口：关键词、分类、价格区间、只看在售四个条件可以组合（REQ-006），商品卡片右下角直接标状态——已预约的商品仍然展示，但点进去不能再次预约，这比直接隐藏更符合“多人竞争同一商品”的真实场景。




图 5-3　商品详情与预约




图 5-4　我的预约（三种状态）

商品详情页把价格、状态、卖家信用、交易须知放在一屏内；底部固定“立即预约”，预约成功后商品转“已预约”，页面上的按钮随之失效。交易须知里写明 24 小时确认与 24 小时下单两个时限，这两个时限在后续三个界面里都会以倒计时出现。

“我的预约”按待卖家确认、已确认、已结束分组。卡片上有倒计时、联系方式可见性（绿色圆点表示当前可见）和操作按钮；已失效的预约里明确写“联系方式已回收”，对应 REQ-012 的可见性规则。




图 5-5　订单详情（已下单）




图 5-6　消息中心

订单详情用时间线呈现“卖家确认 → 买家下单 → 等待交付”三个阶段，订单号、下单时间、卖家联系方式都在一屏内；底部“取消下单”对应 REQ-026，并提示取消会记违约。

消息中心列出四类站内通知：卖家确认、预约即将超时、商品被下架、操作记录已生成。通知内容在业务事务内落库，所以“作废”和“通知”不会出现一个有一个没有的情况（对应第四章 ADR-05）。

## 5.3 卖家端界面




图 5-7　我的商品（按状态分组）




图 5-8　发布 / 编辑商品

卖家商品页顶部是四个数字（在售、待审核、待处理预约、累计成交），下面按状态分组。不同状态给不同操作：已驳回显示驳回理由和“修改并重新提交”，待审核只能等，已发布可以调价、下架，已预约直接给“去确认”。这正是状态转换表在界面上的投影（REQ-029）。

发布页只做三件事：图片、描述、分类与价格。底部“保存草稿”和“提交审核”分开，并在页面上提前告知“驳回会记违约一次”，把后果说在操作之前（REQ-003、004、018）。




图 5-9　卖家预约与订单管理

预约管理页把待确认的预约按剩余时间排序，卡片上直接显示买家联系方式（预约有效期内可见）、剩余确认时间和“确认这笔预约”。确认之后卡片变成“已确认 · 等待买家下单”，并显示买家的下单倒计时。这一屏是 REQ-009、REQ-012 在界面上的落点。

## 5.4 管理员端界面

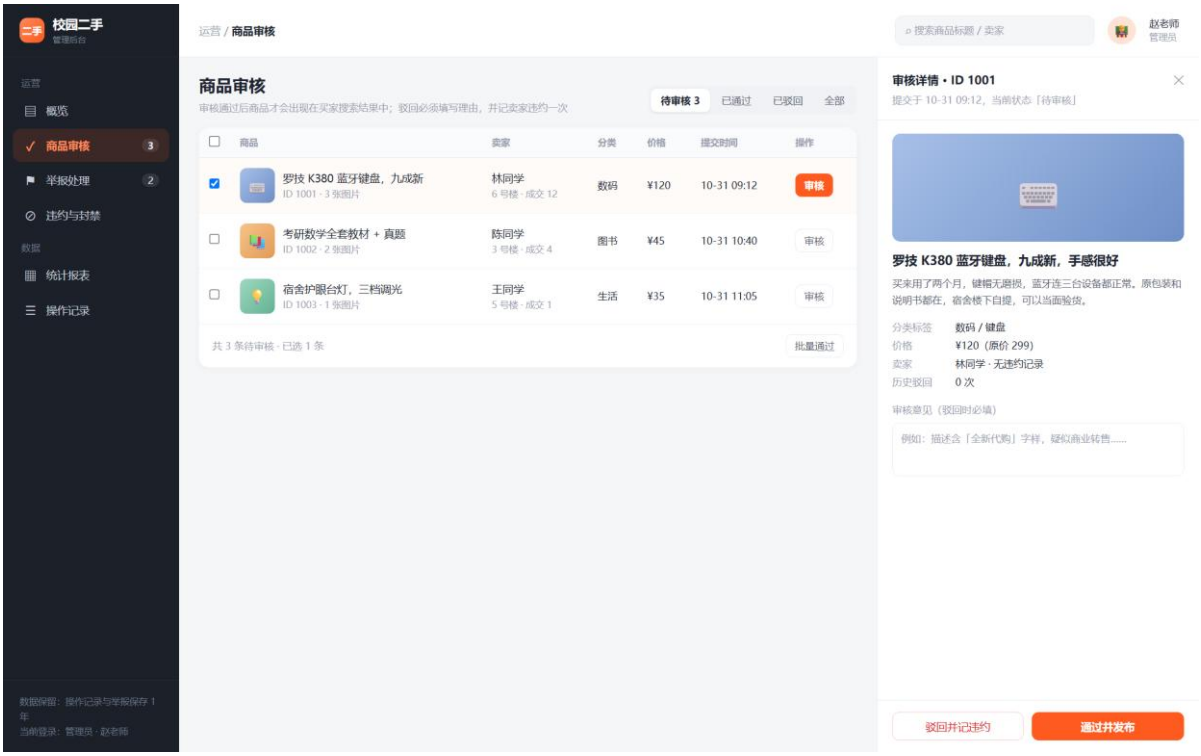



图 5-10　管理后台：商品审核

审核台左边是待审核列表，右边是详情抽屉：商品图、描述、分类、价格、卖家违约历史一次看全。驳回必须填理由（ReasonGuard 在界面上的对应），通过后商品直接变“已发布”。表格支持勾选批量通过，为后续的批量操作留了位置。

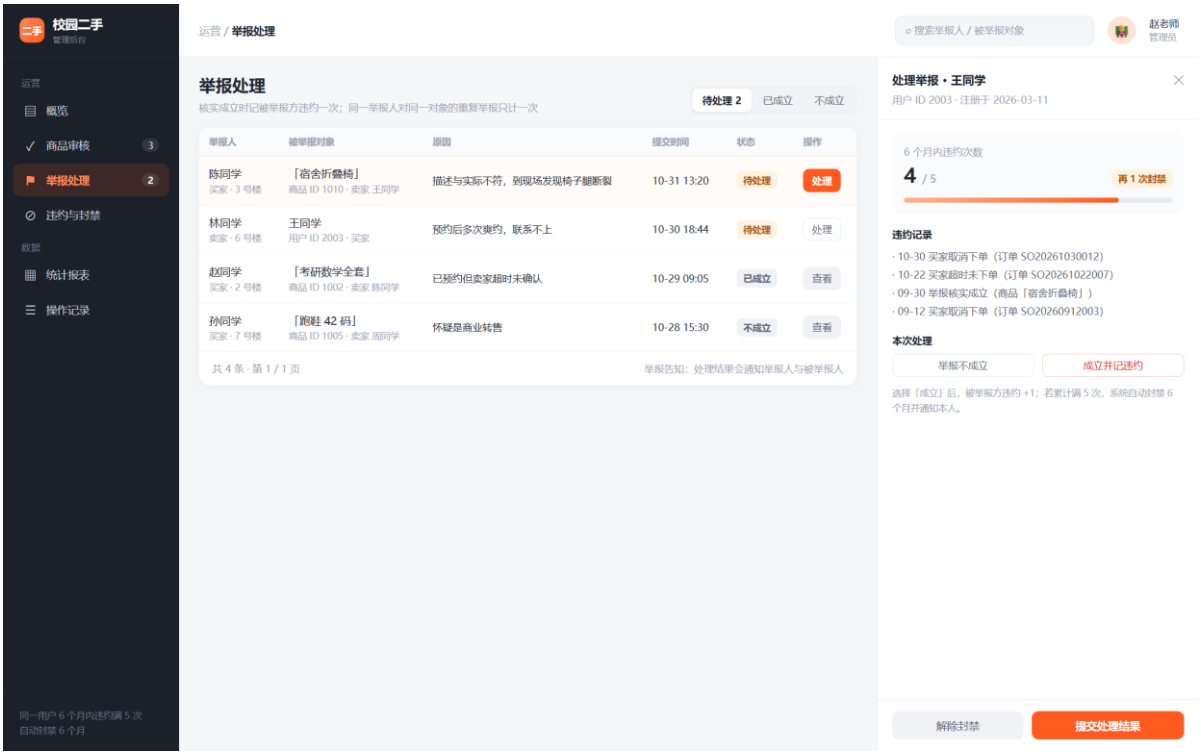



图 5-11　管理后台：举报处理与违约封禁

举报列表展示举报人、被举报对象、原因和状态。右侧抽屉把“这个用户 6 个月内违约 4 次”做成进度条，处理成立时违约 +1，满 5 次由系统自动封禁 6 个月，并通知本人（REQ-013、REQ-018）。

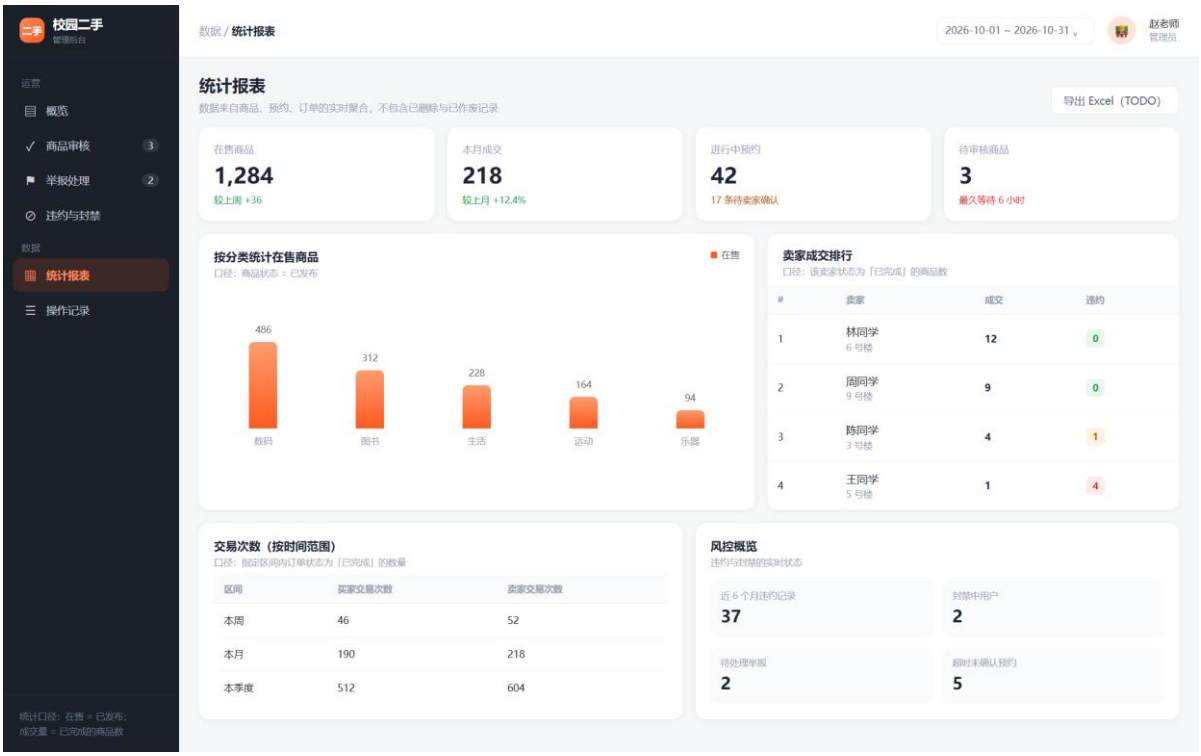



图 5-12　管理后台：统计报表

统计页按 REQ-016 的三个口径出数：按分类统计在售商品、按卖家统计成交量、按时间范围统计交易次数，每个图卡下方都标了口径（在售 = 已发布；成交量 = 已完成），避免“数字对不上”的争论。右下角的风控概览把违约、封禁、待处理举报、超时预约放在一起，管理员一眼能判断今天要不要处理。

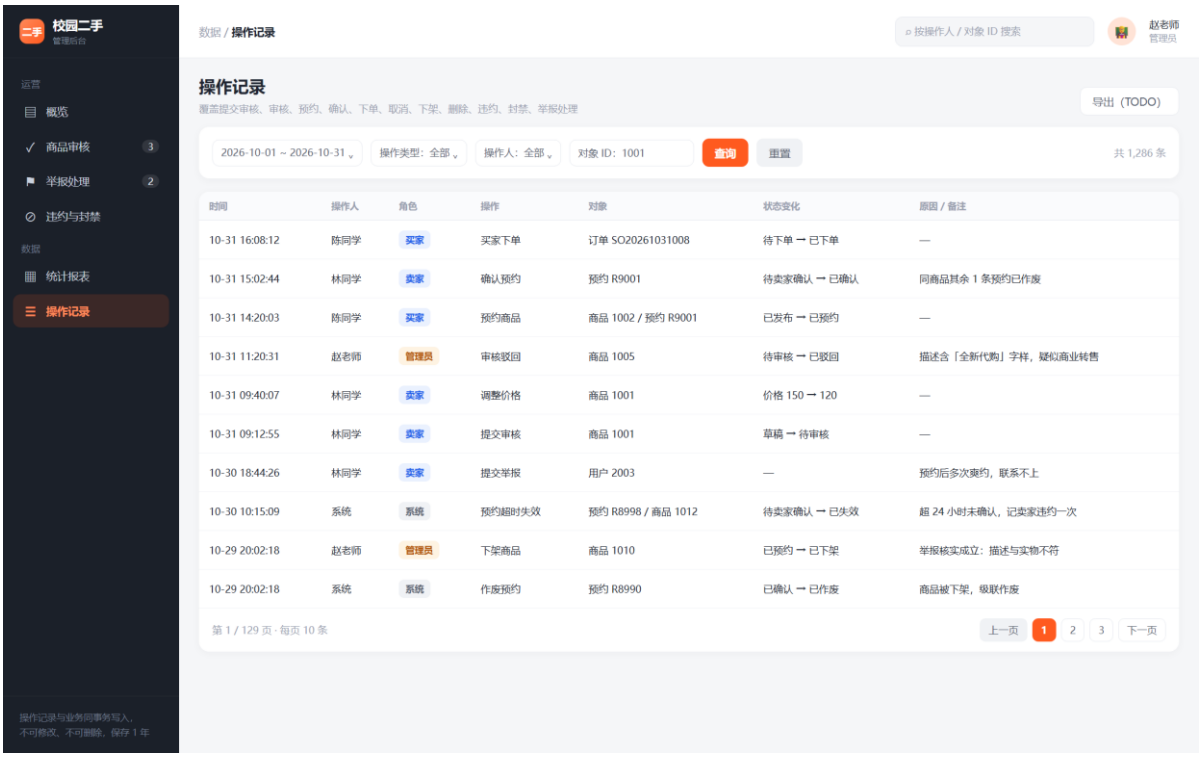



图 5-13　管理后台：操作记录查询

操作记录页按时间范围、操作类型、操作人、对象 ID 组合查询，表格里专门有“状态变化”一列（如“已预约 → 已下架”），原因列记录管理员填写的依据。对应 REQ-027 的“可查询、不可修改、保存至交易完成后 1 年”。

## 5.5 异常状态在界面上的呈现

表 5-2　异常状态在界面上的呈现

| 异常场景 | 界面表现 | 对应需求 |
| --- | --- | --- |
| 商品已被预约 | 商品卡片标“已预约”，详情页预约按钮置灰并提示“该商品已被预约” | REQ-008 |
| 卖家 24 小时未确认 | 预约卡片显示剩余时间，超时后变“已失效”并注明“联系方式已回收” | REQ-010、012 |
| 买家超时未下单 | 订单卡片变“已取消”，提示“已记买家违约一次” | REQ-025 |
| 商品审核驳回 | 卖家商品卡标“已驳回”，展开显示驳回理由与“修改并重新提交” | REQ-004 |
| 连续登录失败 5 次 | 登录按钮上方提示锁定 30 分钟，锁定期间正确密码也拒绝 | REQ-019 |
| 管理员下架商品 | 买家消息中心收到“商品已被下架，订单已作废” | REQ-028 |

界面上还有一处没有单独出图：被封禁的卖家进入商品页时，应在顶部出现一条“账号封禁中，暂不能上架或修改商品”的提示条。这条列为实现阶段的 TODO，界面规范与提示文案已经写在设计说明里。

# 六、AI 辅助设计的过程与反思

本次设计允许使用大模型辅助。我的做法是：先自己把实验一的需求基线、非功能约束整理清楚，再让模型生成初稿，然后逐条审查、把不合适的地方改掉，最后用 Git 把“AI 初稿”和“人工修改”分成两次提交。下面记录每一次使用的场景、生成结果、优缺点和修改后的结果。

## 6.1 使用方式概述

表 6-1　五次主要使用

| 轮次 | 使用场景 | 输入 | 得到的东西 |
|---|---|---|---|
| 1 | 架构与模块划分初稿 | 实验一 26 条需求、非功能约束、技术栈 | 5 个模块、8 条关键决策、11 张表的数据设计、部署方案 |
| 2 | 类图与 UML 选型 | 模块清单、状态机要求 | 28 个类的类图（状态模式）、组件图、用例图、时序图 |
| 3 | 并发与超时方案 | REQ-008 唯一预约、REQ-010 超时、REQ-020 环境约束 | 行锁方案、全表扫描 + 同步通知方案 |
| 4 | UI 信息架构与组件 | 三类角色的任务清单 | 移动端信息架构、卡片式列表、状态徽章与倒计时组件方案 |
| 5 | 反向检查 | 初稿的模块依赖表 | 指出 M2 与 M4 可能互相依赖、审计与业务的事务边界问题 |

## 6.2 逐次记录

### 6.2.1 AI-1：架构与模块划分初稿

场景：把实验一的 26 条已确认需求按“角色权限、商品状态、查询统计、并发竞争、审计”分组，连同 REQ-020 的部署约束，一次性交给模型，要求它给出模块划分、模块职责、数据表和关键决策。

生成结果：5 个业务模块（用户与权限、商品、交易、管理、审计与统计），8 条关键决策，11 张表的设计，以及一张单机部署图。整体结构是完整的，模块职责、需求追溯、字段设计都能直接用。

优点：覆盖全面，没有漏掉需求；术语与需求文档一致；按聚合根划分的思路是对的；数据表字段基本可用。

缺点：检索和商品写路径混在一个模块，REQ-022 的 1 万商品查询没有独立的优化空间；违约记录被分到三个模块里记，REQ-018 的“6 个月 5 次”会出现三个计数器；通知没有归属模块；审计用 AOP 切面，拿不到业务的前后状态。

修改优化：拆成 7 个模块，新增检索、信用、通知三个模块；审计改成 M0 定义端口、M7 实现；下架级联改成同事务同步事件，消除 M2 与 M4 的循环依赖。详见《人工修改记录》HM-01。

### 6.2.2 AI-2：类图与 UML 选型

场景：让模型“按面向对象设计给出类图，包含商品状态变化”，并顺带问了几种 UML 图的区别与 PlantUML 写法。

生成结果：28 个类的类图，商品状态用状态模式（抽象类 + 8 个状态子类），关系与多重性基本正确；组件图、用例图、时序图也一并给出。

优点：继承、组合、多重性画得规范；状态类的命名与需求里的 8 个状态一一对应；UML 图类型的解释准确，PlantUML 语法可以直接跑通。

缺点：28 个类挤在一张图里，导出后 5890×2166 像素，A4 上根本看不清；状态模式让 8 个类各实现 7 个方法，真正有效的规则被淹没；用例图 22 个用例被拉成细长条。

修改优化：类图拆成“领域模型”和“服务与模式”两张；状态模式换成 15 条转换表 + 守卫链，因为 REQ-029 的验收方法是逐条对照转换，表比类好核对；用例图按角色拆两张，用例数收敛到 18 个。详见 HM-02、HM-06。

### 6.2.3 AI-3：并发预约方案

场景：把 REQ-008 的并发要求（同一商品只允许一条有效预约、并发只有一个成功、旧页面继续操作要拒绝）交给模型，让它给方案。

生成结果：事务内 SELECT ... FOR UPDATE 锁商品行，再查有无有效预约，没有则插入。附了完整的时序图。

优点：直观易懂，时序图把加锁、校验、插入的顺序画得很清楚；对“先查后插”的竞态解释到位。

缺点：锁的是商品行，同商品的详情查询会和预约事务抢同一把锁，与 REQ-022 的“20 并发查询 1 秒返回”冲突；正确性依赖每个入口都记得加锁，新加一个入口忘了就出两条有效预约。

修改优化：改成 active_key 生成列 + 唯一索引，把唯一性交给数据库，插入冲突即拒绝；预约流程里不再出现行锁。时序图也重画成两个并发请求撞唯一键的版本。详见 HM-03。

### 6.2.4 AI-4：超时处理与通知

场景：REQ-010、REQ-025 要求 24 小时超时自动失效、自动取消，并通知买家、卖家、管理员。

生成结果：@Scheduled 每分钟全表扫描到期记录，处理时直接向通知表插入消息。

优点：代码最短，逻辑直白，单实例下确实能跑；把“扫描—改状态—记违约—通知”四步串得很清楚。

缺点：confirm_deadline 没有索引，数据量一大就是全表扫描；通知写在业务方法里，业务事务回滚后消息已经落库，用户会收到“预约成功”但实际没成功——违反 REQ-027“操作失败不得留成功记录”。

修改优化：加 (state, deadline) 索引、每批 200 条、用带状态条件的 UPDATE 做幂等；通知内容与业务同事务落库，只在提交后推送。详见 HM-04。

### 6.2.5 AI-5：UI 信息架构与组件

场景：让模型根据三类角色的任务清单，给出移动端的信息架构和关键组件（商品卡片、状态徽章、倒计时、审核抽屉）。

生成结果：买家 4 个 Tab、卖家 4 个 Tab、后台 6 项菜单；卡片式列表 + 状态徽章 + 倒计时的组合；审核用“列表 + 右侧详情抽屉”的布局。

优点：信息架构合理，和交易主线对得上；组件划分清楚，状态色能直接映射需求里的状态；后台抽屉比跳新页面更适合连续审核。

缺点：默认套用了电商模板，没有考虑“已预约的商品仍要展示但不能点”的细节；没有体现“驳回必须填理由”；后台缺少违约次数与封禁的联动展示。

修改优化：商品卡片保留状态徽章、详情页把不可预约的按钮置灰并给原因；审核抽屉里把驳回理由做成必填项；举报抽屉里加“6 个月内违约 4/5”的进度条和封禁提示。

## 6.3 人工修改的五处关键决策

表 6-2　五处人工修改

| 编号 | 决策点 | AI 原方案 | 改后 | 主要理由 |
|---|---|---|---|---|
| HM-01 | 模块划分 | 5 个模块 | 7 个模块（检索、信用、通知独立） | 检索性能、违约口径、通知复用 |
| HM-02 | 商品状态机 | 状态模式（8 个状态类） | 15 条转换表 + 守卫链 | REQ-029 要逐条验收，表可对照 |
| HM-03 | 唯一有效预约 | SELECT ... FOR UPDATE 行锁 | active_key 生成列 + 唯一索引 | 读路径不能被写锁阻塞；约束应由数据库保证 |
| HM-04 | 超时与通知 | 全表扫描 + 业务方法内同步写通知 | 索引分批 + 幂等更新 + 同事务落库、提交后推送 | 性能与事务一致性 |
| HM-05 | 联系方式可见性 | 可见性表，状态变化时写/删 | 实时推导，不建表 | 七条状态路径都要同步，漏一条就出错 |

## 6.4 反思：AI 在设计里能做什么、不能做什么

- 它能很快把需求“铺开”成结构：模块、表、字段、图都能在几分钟内给出一个像样的初稿，省下的是“从零开始搭骨架”的时间，这部分大约占整个设计的一半工作量。
- 它给的是“教科书答案”，不是“你这个项目的答案”。状态模式、悲观锁、全表扫描、AOP 审计，单看每一条都没错，但放到“单实例、无中间件、要逐条验收”的项目里就不合适。判断合不合适，只能人来。
- 它不判断“这个约束该由谁保证”。唯一有效预约是用锁还是用唯一索引，本质是“把正确性放在应用层还是数据库层”的取舍，模型倾向于给应用层的方案，因为它更“面向对象”，但这恰恰是最容易漏的地方。
- 生成物看起来完整，不代表内部一致。初稿里 M2 和 M4 互相依赖、通知与事务边界不一致、类图大到看不清，这些都不是“写错了”，而是“没有站在评审者角度检查”。
- 图的生成成本很低，但“图能不能被看懂”是另一回事。16 张图里，类图和用例图都重画过，原因是第一版塞了太多东西——这一点模型不会提醒你。

- 最有用的一次用法其实是“反向检查”：把依赖表发给模型，让它找循环依赖和事务边界问题，它确实指出了 M2/M4 的问题。让模型当“审稿人”比当“作者”更可靠。

## 6.5 能不能设计一套“人只做检查校对”的工作流

按这次的实际体验，答案是可以把大部分“产出”交给模型，但有三类判断必须留在人这边。我试着把流程写成六步：

5. 准备上下文：把需求基线、非功能约束、技术栈、术语表整理成一份固定输入，每次生成都带上，避免模型自由发挥。
6. 分块生成：一次只让它做一件事（先模块、再类图、再时序图、再 UI），不要一次要一整套，否则错了很难定位。
7. 规则化自检：用一份固定清单逐项打勾——模块是否覆盖全部需求、依赖是否有环、每个状态转换是否在表里、事务边界是否覆盖级联、界面是否覆盖异常态。清单里的项目都是可以客观判断的。
8. 人工决策点：把模型给的方案和备选方案并列，由人拍板。这次的五处修改（模块、状态机、锁、超时通知、可见性）都属于这一类。
9. 回归验证：改完之后回头看图和表是否还一致，比如改了锁方案，时序图、类图、数据表三处都要同步。
10. 版本留痕：AI 初稿和人工修改分两次提交，diff 就是审查记录。

三个必须人工的卡点：一是“约束该由谁保证”的取舍（应用层还是数据库层），二是“验收口径能不能对上需求”的判断，三是“这张图评审时能不能看懂”的判断。前两个错了会在实验三返工，第三个错了会在答辩时被问住。除此之外的重复劳动——画图、排版、写字段、生成初稿文字——确实可以交给模型，人只需要检查和校对。

所以我的结论是：这套工作流可以把设计阶段的体力活压缩到原来的三分之一左右，但“检查校对”本身不是被动劳动，要能看出问题，前提是自己先把需求和约束想清楚。模型负责“给选项”，人负责“做选择并为选择负责”，这条边界在这次实验里体现得很明显。

# 七、Git 版本记录

全部设计与报告都在实验一使用的仓库里追踪，仓库地址 https://github.com/qiaoyi1234/SE-project.git，分支 main。为了让“AI 生成”和“人工修改”能分开看，实验二的提交按下面的顺序组织：

表 7-1　实验二的提交组织

| 提交 | 内容 | 说明 |
| --- | --- | --- |
| chore: 调整仓库结构 | 实验一移入 第一次实验/，实验二放在 第二次实验/ | 纯重命名，4 个文件 100% rename，内容零改动 |
| feat(lab2): AI 生成设计初稿 | 架构设计.md（5 模块）、类图、组件图、用例图、时序图 | AI 初稿，对应第六章的 AI-1、AI-2 |
| refactor(lab2): 人工修改版设计与建模 | 7 模块、状态转换表、唯一索引、超时分批、可见性实时推导；16 张 UML；13 屏 UI | 五处决策的落地，对应《人工修改记录》HM-01～HM-05；实验二设计基线 |
| docs(lab2): 实验报告 | 本报告 PDF 与生成脚本 | 在基线之后单独提交，不改变设计基线的内容 |

实验二设计基线的完整提交哈希为：

```
944f74bb810df2545f8974082ec74b35ee76a901
```

该提交打了标签 lab2-design，包含全部设计文档、16 张 UML 与 13 屏 UI。本报告本身不在这个提交里——报告在基线之后单独提交，这样报告里写的哈希不会因为“报告自己也在提交里”而算不出来，这也是实验一采用过的做法。

初稿提交之后，任何一处修改都能通过 git diff 看到。以架构设计文档为例，初稿里的“5 个业务模块”“状态模式”“行锁”“可见性表”在人工修改版里分别变成了“7 个模块”“转换表 + 守卫”“唯一索引”“实时推导”，diff 与《人工修改记录》里的说明一一对应，没有出现“报告说改了、代码没改”的情况。

## 7.1 怎么复现这些图

UML 目录下每个 PNG 都有同名 .puml 源码，用 PlantUML 重新渲染即可：

```
java -jar plantuml.jar -tpng -charset UTF-8 UML/*.puml
```

渲染需要 Graphviz 提供布局（本机使用 Graphviz 16.1.0），中文使用“Microsoft YaHei”字体，所以源码里统一写了 skinparam defaultFontName。UI 原型是 HTML/CSS 写的，用无头 Chrome 截图生成 PNG，源码在 第二次实验/ui_src/ 下，截图脚本是 build/shot_ui.ps1。

# 八、提交清单与要求对照

## 8.1 提交物结构

压缩包命名为 实验 2_241220173_乔轶.zip，解压后的目录结构如下：

```
实验 2_241220173_乔轶/
├── 实验报告.pdf
├── UML/
│   ├── 01_UML 类图-领域模型.png / .puml
│   ├── 02_UML 类图-服务与模式.png / .puml
│   ├── ...（共 16 张，PNG 与 PUML 同名）
│   └── 16_UML 时序图-超时扫描.png / .puml
└── UI 界面/
    ├── 界面 01_登录注册.png
    ├── ...（共 13 屏）
    └── 界面 13_操作记录查询.png
```

## 8.2 与实验要求的对照

表 8-1　与实验要求的对照

| 实验要求 | 本报告的位置 | 完成情况 |
|---|---|---|
| 说明所选项目 | 1.1 节 | 二手交易系统，沿用实验一需求基线 |
| 软件的主要功能 | 第二章（角色权限、7 个模块、三条主流程、需求覆盖） | 完成 |
| 软件架构 UML 图，写明模块划分（含实验一基本要求模块） | 表 2-2、3.5 节、图 4-5 | 完成，含六项基本要求的逐条对应 |
| 描述模块之间的逻辑关系及设计原因 | 3.3、3.4、3.6、4.5 节 | 完成，含依赖表与两处反向解耦的理由 |
| 类图为必画图 | 图 4-1、图 4-2 | 完成（拆成领域模型与服务模式两张） |
| 结构图、行为图、交互图每类至少一种 | 表 4-1：结构 6 张、行为 7 张、交互 3 张 | 完成，共 16 张 |
| 每个功能模块的 UI 界面 + 文字说明 | 第五章，13 屏，覆盖 7 个模块 | 完成 |
| AI 使用记录（场景、生成结果、优缺点、修改优化后结果） | 第六章 6.1–6.3 | 完成，5 次记录 + 5 处决策 |
| 总结大模型如何辅助设计、能否设计人只做检查校对的工作流 | 6.4、6.5 节 | 完成，给出六步流程与三个人工卡点 |
| 使用 Git 追踪 AI 生成与人工修改 | 第七章 | 完成，初稿与人工修改分两次提交 |
| UML 图使用 PlantUML 绘制 | UML/*.puml 与 PNG 同名 | 完成，源码随包提交 |

## 8.3 遗留问题与后续计划

按照实验说明，实验二只要求原型级设计，具体实现留给实验三。目前明确留作 TODO 的有：被封禁卖家进入商品页时的限制提示条、统计报表导出 Excel、管理员批量审核的异步反馈、站内信的 SSE 实时推送、数据库自动备份脚本。这些不影响架构完整性，在实验三实现时按第五章的界面规范和第三章的接口约定补齐即可。

另外，实验三的实现如果与本次设计冲突，按课程说明属于正常现象；届时会以实现为准回头更新设计文档与 UML，并在报告中说明差异原因。